



GH-600 Practice Exam

Developing in Agentic AI Systems — Practice Mode

100 questions · Friday, May 22, 2026

Correct answers highlighted · Full explanations included

Questions & Answers

Q1

Domain B: MCP/tool use and environment permissions

code or config artifact

hard

A mobile backend repository tries to customize Copilot cloud agent's setup workflow as shown. Copilot does not use the intended environment. Which correction should the team make first?

The repository administrator wants the setup to run before each Copilot task without relying on unsupported runner settings.

 .github/workflows/copilot-setup-steps.yml

```
name: Copilot setup
on:
  workflow_dispatch:
jobs:
  prepare-agent:
    runs-on: macos-latest
    timeout-minutes: 90
    steps:
      - uses: actions/checkout@v6
      - run: brew install swiftlint
      - run: swiftlint --version
```

- A Keep macOS and add a second Linux job that installs the same tools for validation.
- B Increase timeout-minutes so slow package installation can finish before Copilot starts.
- C Move the workflow into .github/agents so it is loaded with the custom agent profile.

D Use a copilot-setup-steps job on Ubuntu or Windows and set timeout-minutes to 59 or less. ✓

✓ **Why the correct answer is right**

The setup workflow must include the required copilot-setup-steps job. Copilot cloud agent supports Ubuntu x64 Linux and Windows 64-bit runners, not macOS, and timeout-minutes has a documented maximum of 59 in this file.

✗ **Why the other options are wrong**

- **A:** Adding another job does not fix the unsupported macOS job or the required setup job name.
- **B:** The documented maximum is 59 minutes, so increasing the timeout moves further from supported configuration.
- **C:** Agent profiles and setup workflows are different mechanisms stored in different paths.

💡 When setup-step artifacts include runners, verify both the required job name and the limited job settings Copilot actually honors.

copilot-setup-steps

runner-compatibility

environment-constraints

Q2**Domain A: Prepare agent architecture and SDLC processes****matching magnet****hard**

A compliance team asks how GitHub can help reconstruct why an agent made a specific change. Match each traceability need to the GitHub Copilot or GitHub artifact that best satisfies it.

The organization wants evidence available during pull request review and later audits without relying on screenshots or chat transcripts stored outside GitHub.

Correct matches:

Identify the human who initiated a Copilot cloud agent change.

→ **Human listed as co-author on the Copilot-authored commit**

Review the tools and reasoning used during an agent session.

→ **Copilot session logs showing reasoning and tool usage**

Verify that the proposed code came from Copilot cloud agent.

→ **Signed commits that appear as Verified on GitHub**

Connect a particular commit to the underlying session evidence.

→ **Commit message links to the session log**

Choices: Signed commits that appear as Verified on GitHub, Commit message links to the session log, Human listed as co-author on the Copilot-authored commit, Copilot session logs showing reasoning and tool usage, A team-maintained spreadsheet of agent assignments, A local terminal history file from the reviewer workstation

✓ Why the correct answer is right

Copilot cloud agent commits are authored by Copilot, list the initiator as co-author, are signed and Verified, and include links from commit messages to session logs containing reasoning and tool-use evidence.

💡 Traceability questions favor first-party artifacts embedded in GitHub: commit metadata, verification, PRs, checks, and session logs.

auditability

traceability

session-logs

Q3**Domain E: Multi-agent orchestration and incident response**

multi select

very_hard

A deprecated incident-remediation custom agent is being retired because a safer replacement is ready. Several old PRs and session logs may be needed for an upcoming audit. Select TWO actions that best preserve workflow continuity and auditability.

New tasks should use the replacement agent, while historical evidence about the deprecated agent must remain traceable.

- A Rewrite old PR descriptions to remove references to the deprecated agent.
- B Commit the removal or rename of the old agent profile with a clear retirement rationale. ✓**
- C Keep both agents automatically invocable until the audit has fully completed.
- D Delete historical session logs so reviewers focus on the replacement agent's behavior.
- E Route new work to the replacement while preserving existing PRs, commits, and log links. ✓**

✓ **Why the correct answer is right**

A versioned configuration change with a retirement rationale documents lifecycle intent. Preserving PRs, commits, and session log links keeps historical work auditable while new work moves to the replacement.

✗ **Why the other options are wrong**

- **A:** Removing references from old PRs weakens traceability and can look like history rewriting.
- **D:** Deleting logs directly conflicts with the requirement to support the upcoming audit.
- **C:** Keeping a deprecated agent automatically invocable increases the chance of new work using it.

💡 Retirement questions are about versioned change control and preserved evidence, not erasing old artifacts.

agent-retirement

audit-continuity

lifecycle-management

Q4**Domain B: MCP/tool use and environment permissions**

multi select

hard

A platform engineer is writing a review checklist for MCP configurations submitted by repository administrators. Select THREE checklist rules that correctly reflect Copilot cloud agent MCP behavior.

The checklist must prevent common validation errors before configurations are saved in repository settings.

- A Approve remote MCP OAuth flows when the server presents a browser consent URL.
- B Permit unprefixed Actions secrets when the repository workflow already uses them.
- C Require url for remote http or sse MCP server configurations. ✓
- D Reject MCP resources and prompts because cloud agent supports MCP tools only. ✓
- E Allow variable substitution in every string field, including tools and type values.
- F Require command and args for local MCP servers that start a separate process. ✓

✓ **Why the correct answer is right**

Copilot cloud agent supports MCP tools, not MCP resources or prompts. Local MCP servers require command and args, while remote MCP servers require a url for the server endpoint.

✗ **Why the other options are wrong**

- E: The documentation excludes tools and type from the fields that support variable substitution.
- A: Remote MCP servers that use OAuth authentication and authorization are not currently supported by Copilot cloud agent.
- B: MCP secrets must be Agents secrets or variables with names prefixed COPILOT_MCP_, not ordinary Actions secrets.

💡 Checklist-style items test exact boundaries. Be cautious with options that generalize a rule, such as substitution applying to all fields.

mcp-schema

local-remote-mcp

configuration-validation

Q5**Domain E: Multi-agent orchestration and incident response**

code or config artifact

very_hard

A production support team defines an incident summarizer custom agent. It should be selected manually after an incident, read repository context, and produce a written summary without editing files or executing commands. Which configuration change best aligns the agent with that intent?

The current profile was copied from an implementation agent and is being reviewed before it is merged to the default branch.

 .github/agents/incident-summarizer.agent.md

```
---
name: incident-summarizer
description: Summarizes production incidents, linked issues, and related pull requests
tools: ["read", "search", "edit", "execute"]
infer: true
---
```

Create a concise incident timeline, identify likely contributing changes, and list follow-up questions for humans.

- A Keep the tool list and add prompt text saying not to edit or execute commands.
- B Remove edit and execute, and use disable-model-invocation true for manual selection. ✓**
- C Change the target to vscode so GitHub.com sessions cannot use this profile.
- D Remove the description so automatic model invocation is less likely to match it.

✓ Why the correct answer is right

For custom agent profiles, tools should be explicitly scoped, and disable-model-invocation true is the current configuration for preventing automatic use while preserving manual selection.

✗ Why the other options are wrong

- **A:** Prompt guidance does not remove write or execution capabilities from the agent profile.
- **D:** Removing the description weakens usability and does not provide a reliable invocation control.
- **C:** Changing the target removes the agent from the intended GitHub Copilot cloud agent workflow.

💡 On agent profile questions, separate behavior instructions from enforceable configuration: tool lists and invocation flags carry more weight.

agent-profile

least-privilege

manual-selection

Q6**Domain A: Prepare agent architecture and SDLC processes**

sequence order

hard

A developer assigns an issue to Copilot cloud agent, then the product owner clarifies an acceptance criterion after the assignment. In which order should the team handle the change with the least rework?

The team knows the agent receives the issue content and existing comments when assigned, but the clarification arrives after assignment.

Correct order:

1. Decide whether the clarification changes scope enough to stop or restart the session.
2. Wait for Copilot to raise the pull request for the assigned issue.
3. Add the clarification as a pull request comment directed to Copilot.
4. Review the agent's revised commits, tests, and updated PR description.

✓ **Why the correct answer is right**

The team should first decide whether the late clarification invalidates the task. If it is still the same task, they should wait for the PR and then provide the new requirement as PR feedback so Copilot can iterate.

✗ **Why the other options are wrong**

- **B-before-A:** A pull request comment cannot be used until Copilot has opened the pull request.
- **C-before-B:** There are no revised commits to review until the agent receives the clarification and iterates.
- **A-before-D:** If the clarification changes the task materially, waiting for the original PR may waste time.

💡 Remember the timing detail: after assignment, new issue comments are not the best way to update Copilot; use the PR once it exists or restart if scope changed.

agent-invocation

requirements-change

pull-request-iteration

Q7

Domain A: Prepare agent architecture and SDLC processes

matching magnet

hard

A solution architect is documenting the boundaries of an agent workflow for a regulated repository. Match each workflow activity to the phase where it belongs.

The architect wants reviewers to distinguish analysis from file-changing behavior and from independent evaluation signals.

Correct matches:

Summarize the issue, identify impacted files, and propose ordered steps. → Planning

Use repository context to explain trade-offs and assumptions behind the approach. → Reasoning

Modify files, run tools, commit changes, and create a pull request. → Action

Run CI, inspect test results, review diff, and request revisions if needed. → Evaluation

Choices: Reasoning, Planning, Action, Evaluation, Long-term memory, Repository provisioning

✓ Why the correct answer is right

Planning defines the proposed steps, reasoning explains why those steps are appropriate, action changes the repository, and evaluation uses tests, checks, and review to judge the result.

💡 Boundary questions test whether you can keep thinking, proposing, doing, and validating separate. File writes usually mark the start of action.

planning-reasoning-action-boundaries

structured-workflow

evaluation

Q8**Domain D: Evaluation, telemetry, error analysis, and tuning**

single choice

hard

A Copilot cloud agent creates a pull request that adds a new file-upload endpoint. The session log shows a CodeQL security issue was detected during the agent's validation pass, but the final pull request still contains the vulnerable pattern. What is the best evaluation and tuning response?

The team wants to keep using the agent for similar endpoint work, but security policy requires vulnerable patterns to be addressed before merge.

- A Merge the pull request after filing a follow-up issue because CodeQL already documented the finding.
- B Treat the finding as a failed evaluation signal and steer Copilot to fix it before review completion. ✓**
- C Approve the pull request if the endpoint tests pass and the vulnerability is not currently exploitable.
- D Disable CodeQL for agent branches to reduce noise and rely on human review instead.

✓ **Why the correct answer is right**

A security finding in the agent's validation path is an evaluation failure for the generated change. GitHub documents that Copilot analyzes its generated code for security issues and that session logs show the analysis and actions taken, so the appropriate response is to have the agent remediate before merge.

✗ **Why the other options are wrong**

- **A:** Documenting a finding is not remediation and leaves a known vulnerability in the pull request.
- **D:** Disabling the scan removes an important automated evaluation signal instead of tuning the workflow.
- **C:** Passing functional tests does not satisfy a security policy requiring vulnerable patterns to be addressed.

💡 In GH-600, scan findings are not merely informational when they map to the task's success criteria; they are feedback for iteration.

codeql

automated-scanning

agent-iteration

Q9**Domain C: Memory, state, and execution**

multi select

hard

A repository migrated from Mocha to Vitest two weeks ago. A Copilot cloud-agent session now proposes a test change using the old ``npm run mocha`` command, which appears to come from a previously learned repository fact. Select TWO actions that best address stale memory while preserving useful repository learning.

The repository owner wants future Copilot sessions to use Vitest, but does not want to disable Copilot Memory for every user.

- A** Have the repository owner remove the stale repository-level fact if it no longer applies. ✓
- B** Copy the old Mocha command into repository instructions so future sessions see both options.
- C** Verify whether the fact citation still matches the current branch before relying on it. ✓
- D** Keep the proposed change and depend on CI failure to correct the command later.
- E** Disable Copilot Memory for the organization until every historical test command is manually audited.

✓ **Why the correct answer is right**

Copilot Memory repository facts are stored with citations and validated against the current branch before use. Repository owners can review and delete repository-level facts, which is the targeted correction for a stale migration fact.

✗ **Why the other options are wrong**

- **E:** Organization-wide disablement is broader than needed and removes useful memory for unrelated repository facts.
- **B:** Preserving conflicting commands in instructions increases context conflict instead of pruning obsolete information.
- **D:** Waiting for CI is reactive and leaves the agent operating from stale context during the task.

💡 When a memory appears stale, prefer validation and targeted pruning over disabling the feature or adding contradictory context.

stale-context

memory-validation

memory-pruning

Q10

Domain C: Memory, state, and execution

multi select

very_hard

An enterprise wants Copilot to learn repository-specific build and architecture facts during cloud-agent work, but compliance also requires certain migration decisions to remain auditable for one year. The tenant uses organization-managed Copilot subscriptions. Select THREE design choices that satisfy the memory and retention requirements.

Developers with read access should benefit from learned repository facts, while long-term audit records must not depend on Copilot Memory retention.

- A Store annual audit decisions as user-level preferences so they travel with each migration lead.
- B Use write-access users with Copilot Memory enabled to initiate fact-generating repository work. ✓
- C Have an enterprise or organization admin enable Copilot Memory for the managed subscription. ✓
- D Persist one-year migration decisions in an external durable store or committed audit artifact. ✓
- E Ask read-only users to create repository facts during code review because they can see the repository.
- F Depend on unused Copilot Memory entries remaining available until the annual audit window closes.

✓ **Why the correct answer is right**

For organization-managed subscriptions, Copilot Memory is off by default and must be enabled by an admin. Repository facts are created from actions by write-access users with Memory enabled. One-year audit decisions should use external durable storage or committed artifacts because unused Copilot Memory entries are deleted after 28 days.

✗ **Why the other options are wrong**

- **F:** Unused facts and preferences are automatically deleted after 28 days, so they do not satisfy a one-year audit requirement.
- **E:** Repository-level facts are created from actions by users with write access and Copilot Memory enabled.
- **A:** User-level preferences are personal workflow signals, not shared auditable migration records.

💡 For mixed retention requirements, separate adaptive Copilot Memory from compliance-grade durable records with explicit retention.

memory-enablement

retention

external-memory

Q11

Domain A: Agent architecture and SDLC integration

single choice

medium

An engineering manager asks why the team should delegate routine cleanup tasks to Copilot cloud agent through GitHub instead of having developers run local AI sessions and manually push the results. Which reason best supports the GitHub-based agent workflow?

The organization needs reviewers and auditors to understand what the agent did, why it did it, and who started the work.

- A GitHub workflows remove the need for developers to review generated changes before merging.
- B GitHub records agent work through branches, pull requests, signed commits, session logs, and review events. ✓**
- C Local AI sessions provide stronger isolation because they avoid repository-level workflow controls.
- D Local AI sessions reduce audit scope because only final commits are visible to reviewers.

✓ **Why the correct answer is right**

GitHub is the system of record and control plane for agent activity. Copilot cloud agent work is visible through branches, PRs, session logs, and signed commits that link back to the session.

✗ **Why the other options are wrong**

- **C:** Avoiding repository controls reduces governance rather than improving SDLC integration.
- **D:** Showing only final commits loses useful planning and execution context for review and audit.
- **A:** Copilot-generated work still requires human review and normal repository controls before merge.

💡 For observability questions, favor answers that keep agent activity inside GitHub artifacts and logs.

observability

auditability

system-of-record

Q12

Domain B: MCP/tool use and environment permissions

code or config artifact

very_hard

A repository-level custom agent should run in Copilot cloud agent to update release notes from merged pull request summaries. Developers report that it is not selected on GitHub.com, and the profile would not have enough tools if selected. Which change best fixes the profile?

The agent should read repository history, search Markdown files, and edit docs/RELEASE_NOTES.md only.

 .github/agents/release-notes.md

```
---
name: release-notes-writer
description: Writes concise release notes from merged pull request context
target: vscode
tools: []
---
```

Update docs/RELEASE_NOTES.md using merged pull request summaries. Do not edit application source files.

- A** Set target to github-copilot and tools to ["read", "search", "edit"]. ✓
- B** Keep target as vscode and add a prompt saying the agent is intended for GitHub.com.
- C** Set user-invocable to false and rely on automatic model invocation from GitHub.com.
- D** Remove the tools property so the agent has enough tools in every environment.

✓ Why the correct answer is right

The target property controls the environment; github-copilot is the appropriate target for Copilot cloud agent on GitHub.com. An empty tools list disables all tools, so the profile should explicitly allow read, search, and edit for the release-note task.

✗ Why the other options are wrong

- **B:** Prompt text does not override a target value that limits where the agent is intended to run.
- **D:** Removing tools enables all available tools, which is broader than needed for release-note editing.
- **C:** Disabling user invocation does not fix the wrong target or the empty tool list.

💡 When a custom agent does not appear or cannot act, check both environment targeting and tool availability; either one can block the intended workflow.

custom-agent-target

tool-aliases

agent-profile

Q13**Domain D: Evaluation, telemetry, error analysis, and tuning**

multi select

hard

A repository owner reviews a failed Copilot cloud agent task. The requested change was to add validation to one checkout endpoint. The session log shows the agent first planned to update the checkout controller, then called a repository-wide replace command that modified generated clients and unrelated tests. Select TWO root-cause classifications that best fit the evidence.

The final pull request failed CI because generated files no longer matched the checked-in API schema, and reviewers noted that the initial plan was narrower than the actual diff.

- A** Context issue, because the agent had no evidence that generated clients existed in the repository.
- B** Memory issue, because stored repository facts should have prevented all generated-file edits.
- C** Reasoning drift, because the final diff diverged from the agent's original task plan. ✓
- D** Environment issue, because CI failed after the agent pushed changes to its branch.
- E** Tool misuse, because the agent used a broad replace command for a narrow endpoint change. ✓

✓ **Why the correct answer is right**

The broad repository-wide replace is tool misuse because the wrong tool scope caused unintended edits. The mismatch between the narrow plan and broad final diff is reasoning drift that should be identified by comparing plans, logs, and outputs.

✗ **Why the other options are wrong**

- **A:** The evidence says the agent modified generated clients, not that it lacked context about their existence.
- **D:** A CI failure is an observed symptom; the root cause is the agent's broad edit and plan divergence.
- **B:** Copilot Memory may help with repository facts, but the evidence does not show a stale or missing memory caused the failure.

💡 When a stem includes both a plan and a tool trace, classify the failure from the causal evidence, not from the final failing check alone.

failure-analysis

root-cause-classification

session-logs

Q14**Domain D: Evaluation, telemetry, error analysis, and tuning**

code or config artifact

hard

A security team adds dependency review as an evaluation gate for agent-created pull requests. The goal is to block runtime dependencies that introduce high or critical vulnerabilities while still allowing development-only moderate findings for now. What change should the team make to the workflow?

The workflow currently fails inconsistently and reviewers are unsure whether the configuration expresses the intended policy.

 `.github/workflows/dependency-review.yml`

```
name: Dependency Review
on: [pull_request]

permissions:
  contents: read

jobs:
  dependency-review:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v5
      - name: Dependency Review
        uses: actions/dependency-review-action@v4
        with:
          fail-on-severity: moderate
          fail-on-scopes: development, runtime
          allow-licenses: MIT, Apache-2.0
          deny-licenses: GPL-3.0
```

- A** Set fail-on-severity to high, set fail-on-scopes to runtime, and use only one license policy field. ✓
- B** Keep the workflow unchanged and evaluate only the final Dependabot alert count after merge.
- C** Remove dependency review and rely on Copilot code review comments for dependency risk evaluation.
- D** Change fail-on-severity to low and keep both license fields to maximize early detection.

✓ **Why the correct answer is right**

The intended gate is high-or-worse runtime vulnerability blocking, so fail-on-severity should be high and fail-on-scopes should be runtime. The dependency review action documentation also states that allow-licenses and deny-licenses are mutually exclusive.

✗ **Why the other options are wrong**

- **B:** Post-merge alert counts are useful telemetry but do not prevent insecure dependencies entering the default branch.
- **D:** Failing on low severity and development scope is stricter than the stated policy and keeps a conflicting license configuration.
- **C:** Copilot review comments can provide feedback, but dependency review is the specific automated enforcement mechanism for dependency changes.

💡 For scan-as-evaluation items, translate the stated policy into the narrowest enforceable check and remove conflicting configuration.

automated-scanning

dependency-review

evaluation-gates

Q15**Domain D: Evaluation, telemetry, error analysis, and tuning**

multi select

medium

A platform team is evaluating whether a planning custom agent improves implementation quality. Each task should produce a structured plan before code changes, and the team wants to measure whether later pull requests follow that plan. Select TWO artifacts that provide the strongest evidence for this evaluation.

The team wants a repeatable review process for comparing intent, agent actions, and final code changes.

- A The repository star count and number of contributors during the evaluation period.
- B The structured plan artifact, compared with the final pull request diff and commit history. ✓
- C The session log showing tool use, validation steps, and reasons for implementation choices. ✓
- D The number of times developers opened the Agents page during the agent session.
- E The count of emojis and reactions on the agent-created pull request summary.

✓ **Why the correct answer is right**

Plan adherence is evaluated by comparing the structured plan with the final diff and commit history, while session logs show Copilot's approach, tools used, and validation steps. Together they reveal whether execution followed intent.

✗ **Why the other options are wrong**

- **D:** Opening the Agents page is an engagement signal, not evidence of plan adherence or implementation quality.
- **E:** Reactions may reflect sentiment but do not connect the plan to the actual implementation.
- **A:** Repository popularity and contributor count are contextual metrics, not task-level evaluation artifacts.

💡 For evaluation-by-artifact questions, choose artifacts that preserve intent, actions, and outputs in GitHub's system of record.

telemetry

plan-adherence

workflow-artifacts

Q16**Domain F: Guardrails, safety, accountability, and governance****code or config artifact****very_hard**

A governance review finds that a custom Copilot agent for incident triage can call every tool on an internal MCP server. The agent only needs to read incident metadata and link existing issues. What is the best correction?

The MCP server also exposes tools that can close incidents, page on-call engineers, and modify severity labels in the incident system.

 .github/agents/incident-triage.md

```
---
name: incident-triage
description: Reads incident context and prepares GitHub issue updates.
tools: ["read", "search", "edit", "incident-mcp/*"]
mcp-servers:
  incident-mcp:
    type: http
    url: https://incidents.example.com/mcp
    tools: ["*"]
    headers:
      Authorization: Bearer ${ secrets.INCIDENT_TOKEN }
---
Use incident context to summarize customer impact and suggest issue updates.
```

- A** Disable the cloud agent firewall because the MCP server already authenticates every request with a token.
- B** Expose only needed read/link tools in the agent tools list and use an appropriately scoped Agents secret. ✓
- C** Replace the MCP server with screenshots from the incident dashboard attached to each GitHub issue.
- D** Keep incident-mcp/* and add an instruction telling the agent to avoid mutating incident records.

✓ **Why the correct answer is right**

MCP tools can be invoked autonomously without per-call approval, so the agent should allow only the specific read or linking tools needed for the task and use a scoped secret from the supported agent-secret mechanism.

✗ **Why the other options are wrong**

- **D:** Natural-language instructions are weaker than tool allowlisting when mutating MCP tools remain available.
- **A:** The firewall is not a substitute for MCP tool governance, and disabling it increases exfiltration risk.
- **C:** Screenshots reduce automation quality and still do not establish a governed tool permission boundary.

💡 When MCP tools appear in an artifact, inspect the effective tool list. The safest answer usually removes unnecessary tools rather than relying on prompts.

mcp-tool-governance

custom-agents

least-privilege

Q17

Domain B: MCP/tool use and environment permissions

single choice

medium

A team wants Copilot cloud agent to use GitHub issue and pull request context while fixing issues in a single repository. The agent does not need information from other repositories. What should the repository administrator choose?

A developer proposes adding a fine-grained PAT to the GitHub MCP server because it seems more explicit than relying on defaults.

- A** Use the built-in GitHub MCP access scoped read-only to the current repository. ✓
- B** Add an organization PAT with read access to all repositories for simpler future reuse.
- C** Disable the built-in GitHub MCP server and configure a custom local GitHub proxy.
- D** Use a broad PAT but restrict repository writes through branch protection and review rules.

✓ **Why the correct answer is right**

The built-in GitHub MCP server for Copilot cloud agent provides read-only tools by default and uses a token scoped to the source repository. That satisfies current-repository context without introducing broader credentials.

✗ **Why the other options are wrong**

- **B:** An organization-wide PAT expands repository visibility beyond the stated need and increases credential blast radius.
- **C:** A custom proxy adds operational complexity without improving the least-privilege posture for this use case.
- **D:** Branch protection governs merges and some writes, but it does not reduce unnecessary read scope granted to the agent.

💡 If the requirement fits the default scoped GitHub MCP behavior, do not add a broader PAT just because it feels more configurable.

github-mcp

repository-scope

least-privilege

Q18

Domain A: Agent architecture and SDLC integration

single choice

hard

A regulated payments team wants Copilot cloud agent to research a legacy billing module and propose a migration approach before any code changes occur. Developers must approve the approach because the module affects transaction reconciliation. Which workflow best satisfies the requirement?

The team is using GitHub.com agent entry points and wants the approval step to be visible in normal development tooling.

- A Move the planning discussion to chat, then paste the final decision into the pull request afterward.
- B Ask Copilot to research and create a structured plan on a branch, then approve before requesting implementation. ✓**
- C Let Copilot open a pull request immediately, then ask reviewers to ignore the initial implementation commit.
- D Assign the full migration issue to Copilot and depend on branch protection to catch unsafe changes.

✓ **Why the correct answer is right**

Copilot cloud agent can research a repository, create an implementation plan, make changes on a branch, and allow iteration before a pull request is created. That supports a visible plan-before-action checkpoint.

✗ **Why the other options are wrong**

- **D:** Branch protection is important, but it evaluates changes after execution rather than approving a plan first.
- **C:** Opening an implementation PR immediately mixes planning and action, which weakens the required checkpoint.
- **A:** External chat may help collaboration, but it reduces traceability compared with GitHub-hosted plan artifacts.

💡 For planning-boundary questions, prefer workflows that separate research and plan approval from implementation.

planning-execution-boundary

human-approval

structured-plan

Q19**Domain A: Agent architecture and SDLC integration****code or config artifact****very_hard**

A repository administrator reviews a proposal to fix blocked dependency downloads for Copilot cloud agent. The proposal disables the firewall because one private package host is blocked. Which action best meets the runtime boundary requirement with the least additional risk?

The agent only needs access to `packages.internal.example.com` in addition to GitHub and the recommended dependency hosts.

 Proposed Copilot cloud agent network policy

Internet access:

`firewall`: disabled

`recommended_allowlist`: enabled

Reason: `npm install` fails because `packages.internal.example.com` is blocked during Copilot setup.

- A** Disable the recommended allowlist and let package managers retry until they find a reachable mirror.
- B** Move package installation into setup steps because the firewall does not apply there.
- C** Disable Copilot cloud agent for the repository until dependencies are made public.
- D** Keep the firewall enabled and add `packages.internal.example.com` to the custom allowlist. ✓

✓ **Why the correct answer is right**

The firewall limits internet access to reduce data exfiltration risk. Adding the specific required host to the allowlist preserves the boundary while enabling required dependency downloads.

✗ **Why the other options are wrong**

- **A:** Removing the recommended allowlist makes dependency installation less reliable and does not add the needed private host.
- **B:** Setup-step processes are outside the firewall's coverage, so moving access there weakens the intended boundary.
- **C:** Disabling the agent is disproportionate when a scoped allowlist entry satisfies the requirement.

💡 For firewall questions, prefer narrow allowlisting over disabling controls or moving work to a less-governed path.

firewall

runtime-boundaries

data-exfiltration-risk

Q20

Domain B: MCP/tool use and environment permissions

single choice

medium

An organization defines an Agents variable `NPM_REGISTRY_URL` for all repositories. One repository must use a different internal registry for Copilot cloud agent setup steps. What should the repository administrator do?

The team wants the exception to apply only to this repository and not affect other services in the organization.

- A Store the URL as `COPILOT_MCP_NPM_REGISTRY_URL` so setup steps can read it.
- B Create a repository-level Agents variable with the same name and the repository-specific value. ✓**
- C Create an Actions repository variable with the same name and update the setup workflow.
- D Rename the organization variable so the repository can define a different unscoped name.

✓ **Why the correct answer is right**

Agents secrets and variables can be configured at organization and repository levels. When the same name exists at multiple levels, the lower-level repository value takes precedence for that repository.

✗ **Why the other options are wrong**

- **C:** Copilot cloud agent receives Agents secrets and variables, not Actions variables, for its own environment.
- **D:** Renaming the organization variable disrupts other repositories and is unnecessary because precedence handles the exception.
- **A:** `COPILOT_MCP_` values are only available to MCP servers, not general setup-step environment usage.

💡 Secret-scope questions often hinge on the dedicated Agents store and precedence. Do not switch to Actions secrets unless the scenario is Copilot CLI in Actions.

agent-secrets

repository-scope

secret-precedence

Q21**Domain D: Evaluation, telemetry, error analysis, and tuning****code or config artifact****very_hard**

A team uses a planning-only custom agent to produce implementation plans for senior review. In three recent runs, the agent executed build scripts and edited source files before writing the plan. Which configuration change best tunes the agent to the intended evaluation workflow?

The team wants the agent to inspect the codebase, search for patterns, and write a Markdown plan file, but not execute commands or modify application code.

 `.github/agents/implementation-planner.agent.md`

```
---
name: implementation-planner
description: Creates implementation plans for review before coding
tools: ["read", "search", "bash", "edit"]
---
```

You are a planning specialist. Analyze the requested feature, inspect the repository, and create a Markdown implementation plan with risks, assumptions, constraints, phases, and dependencies.

Before finishing, validate that the proposed approach is feasible.

- A** Keep all tools and require a reviewer to inspect the session log after each run.
- B** Keep bash and add an instruction telling the agent to avoid source edits unless reviewers object.
- C** Remove read and search so the agent cannot inspect files before creating the plan.
- D** Remove bash, keep read/search/edit, and specify that edit is only for writing the plan document. ✓

✓ **Why the correct answer is right**

The implementation planner pattern uses read, search, and edit to inspect the repository and create structured Markdown plans. Removing bash narrows tool access to the intended workflow, while constraining edit to the plan file addresses the observed failure.

✗ **Why the other options are wrong**

- **B:** A soft instruction leaves the unnecessary execution tool available and does not directly remove the misuse path.
- **C:** Removing read and search prevents the agent from grounding the plan in repository evidence.
- **A:** Reviewing logs after the fact helps diagnosis but does not tune the agent's tool access.

💡 When tuning tool misuse, remove or constrain the tool that enabled the failure while preserving the minimum tools needed for the task.

Q22

Domain B: MCP/tool use and environment permissions

single choice

medium

A documentation-maintenance team is creating a custom agent that should inspect Markdown, locate broken internal links, and propose documentation-only edits. The repository already has MCP servers configured for issue management and deployment status. Which tool configuration best fits the requirement?

The agent must not run build commands, change application code, or call unrelated external tools during autonomous execution.

- A Omit the tools property so the agent can choose any available built-in and MCP tool.
- B Set tools to ["read", "search", "edit"] and instruct the agent to edit documentation files only. ✓**
- C Set tools to ["read", "search", "execute"] so it can verify links with shell scripts.
- D Set tools to ["*"] and rely on branch protection to catch inappropriate repository changes.

✓ **Why the correct answer is right**

The agent needs repository read/search plus edit capability, but not shell execution or unrelated MCP tools. Custom agent tool lists can explicitly enable only specific aliases, and omitted tools or ["*"] enables all available tools.

✗ **Why the other options are wrong**

- **A:** Omitting the tools property grants all available tools, including configured MCP tools that are not needed for documentation edits.
- **C:** Shell execution adds unnecessary capability for a documentation-only task and increases the execution surface.
- **D:** Branch protection is a review control, not a least-privilege tool configuration for autonomous execution.

💡 For GH-600 tool-scope questions, start from the smallest tool set that can complete the task, then reject options that defer prevention to review controls.

Q23**Domain D: Evaluation, telemetry, error analysis, and tuning**

code or config artifact

hard

A team measures agent quality by requiring the full CI workflow to pass on every agent-created pull request. A recent pull request was merged after reviewers saw only the agent's local test summary, and a deployment regression occurred. What is the best diagnosis and remediation?

The team reviews the pull request timeline and finds the following workflow output.

 Pull request workflow output

Pull request: copilot/fix-cache-invalidation

Author: Copilot

Timeline:

- Copilot pushed commit 4f1a9c2
- pull_request workflow: not started
- reviewer comment: "Agent says unit tests passed locally"
- pull request merged

Actions note:

Workflows from this actor were not run automatically for this pull request.

- A** Classify this as a reasoning error because the agent chose an incorrect cache invalidation strategy.
- B** Replace local tests with Copilot code review comments as the primary merge signal.
- C** Tune Copilot Memory so future summaries include more detail about locally executed tests.
- D** Classify this as an evaluation gap and require CI workflow evidence before merge. ✓

✓ **Why the correct answer is right**

The stated quality gate was full CI, but the workflow never ran and reviewers accepted a local-test summary. GitHub documents that Actions workflows do not run automatically by default when Copilot pushes changes to a pull request, so the remediation is to require CI evidence or explicitly manage workflow execution before merge.

✗ **Why the other options are wrong**

- **A:** The artifact does not show the agent's implementation reasoning; it shows a missing evaluation signal.
- **C:** A more detailed local summary still would not satisfy the required CI workflow gate.
- **B:** Copilot code review can provide feedback, but it is not a substitute for the required workflow validation.

💡 Do not confuse an agent's self-reported validation with an enforced GitHub check. Evaluation evidence should come from the required artifact or status check.

workflow-artifacts

ci-validation

failure-analysis

Q24**Domain E: Multi-agent orchestration and incident response**

multi select

hard

A platform team is building a dashboard for multi-agent incident repair sessions. The dashboard must reconstruct which sub-agent ran, whether it succeeded, and which execution span produced each result. Select TWO telemetry design choices the team should implement.

Several instances of the same custom agent may run during one parent session, and reviewers need a post-hoc execution tree.

- A Suppress selected and deselected events because only final PR status affects review.
- B Group all events only by agent display name to simplify the dashboard model.
- C Treat a completed event as proof that the resulting code is safe to merge.
- D Stream sub-agent lifecycle events from the parent session into a durable review artifact. ✓
- E Correlate started, completed, and failed events by toolCallId when building the execution tree. ✓

✓ **Why the correct answer is right**

The lifecycle events include toolCallId on started, completed, and failed events, which lets the dashboard correlate spans. Persisting streamed events creates an artifact suitable for audit and post-hoc analysis.

✗ **Why the other options are wrong**

- **B:** Agent names are not unique execution spans, especially when the same agent runs more than once.
- **A:** Selection and deselection events help explain delegation and context switches during the session.
- **C:** Successful execution is an operational signal, not a substitute for code review and validation.

💡 Observability questions usually reward correlation IDs and durable artifacts, not summaries that collapse execution detail.

observability

subagent-events

audit-trail

Q25

Domain F: Guardrails, safety, accountability, and governance

single choice

hard

A financial services organization must prove which human initiated a Copilot cloud agent change, why the agent made the change, and whether agent activity occurred outside normal hours. Which accountability design best satisfies the requirement?

The organization already requires pull request review, but auditors need evidence after merge for quarterly controls testing.

- A Create a bot that labels Copilot pull requests and exports the labels to a spreadsheet each month.
- B Use Copilot-authored signed commits, session-log links, and enterprise audit-log queries or streaming for agent activity. ✓**
- C Ask reviewers to add a signed comment confirming they read the diff and approve the agent behavior.
- D Require developers to paste a summary of every Copilot session into the pull request description before merge.

✓ **Why the correct answer is right**

Copilot cloud agent commits identify Copilot as author, mark the human initiator as co-author, include session-log links, and appear signed. Enterprise audit logs can be queried for Copilot and agent activity and streamed for longer-term monitoring.

✗ **Why the other options are wrong**

- **D:** Manual summaries are useful context but are weaker evidence than system-generated commit, session, and audit records.
- **A:** Labels help filtering but do not show who initiated work, why a change was made, or detailed activity history.
- **C:** Reviewer comments document approval, but they do not replace agent session logs or enterprise audit events.

💡 Accountability questions favor system-of-record evidence: signed commits, session logs, and audit logs. Manual notes are usually supplemental.

accountability

auditability

session-logs

Q26**Domain A: Prepare agent architecture and SDLC processes****sequence order****very_hard**

An enterprise platform team wants to introduce a custom database-migration agent but must prove it is safe before broad availability. In which order should the team run the pilot and release process?

The agent will eventually be available organization-wide, but early tests must be limited to a small set of maintainers and repositories.

Correct order:

1. Create or prepare the .github-private repository with restricted visibility for the pilot.
2. Author the agent profile with scoped instructions, tools, and migration boundaries.
3. Test the agent on representative low-risk tasks and review session evidence.
4. Move the validated profile into the organization release location and document usage rules.

✓ **Why the correct answer is right**

A safe rollout prepares the controlled repository first, then authors the scoped agent profile, then validates behavior with representative tasks, and only then releases the profile for wider organization use with guidance.

✗ **Why the other options are wrong**

- **B-before-A:** Authoring can happen locally, but controlled organization release requires the correct repository and visibility foundation.
- **D-before-C:** Publishing before representative testing expands risk before evidence exists.
- **C-before-B:** There is no meaningful pilot until the custom agent profile has scoped behavior to test.

💡 Pilot sequencing items reward controlled exposure first and broad release last. Avoid options that publish before validation.

custom-agent-pilot

release-governance

least-privilege-rollout

Q27

Domain A: Prepare agent architecture and SDLC processes

matching magnet

hard

A platform review finds that several autonomous agent runs completed with confusing results: some errors were hidden, some tasks changed scope mid-run, and some outputs were hard to review. Match each observed behavior to the best architectural mitigation.

The team wants to reduce operational risk without disabling agent workflows across the organization.

Correct matches:

The agent catches tool failures and continues without reporting degraded output.

→ **Add escalation paths that surface failures in logs, PR comments, or checks**

The agent keeps applying unrelated historical context from earlier work items.

→ **Scope memory and context to the current task and prune stale information**

The agent produces only narrative summaries instead of reviewable artifacts.

→ **Require structured outputs such as plans, PR descriptions, and test evidence**

The agent attempts high-risk changes without a decision checkpoint.

→ **Insert human approval before irreversible or compliance-sensitive actions**

Choices: Require structured outputs such as plans, PR descriptions, and test evidence, Add escalation paths that surface failures in logs, PR comments, or checks, Scope memory and context to the current task and prune stale information, Insert human approval before irreversible or compliance-sensitive actions, Increase token limits so the agent can retain more repository history, Move the work outside GitHub to reduce pull request noise

✓ **Why the correct answer is right**

The mitigations map to the underlying failure modes: hidden failures need escalation, context leakage needs scope control, unreviewable prose needs structured artifacts, and risky autonomy needs approval checkpoints.

💡 Matching questions often pair symptoms with controls. Identify whether the failure is about visibility, context, output structure, or autonomy.

agent-anti-patterns

mitigation

observability

Q28**Domain E: Multi-agent orchestration and incident response**

code or config artifact

very_hard

A team notices that its multi-agent incident dashboard sometimes marks the wrong worker as failed when two instances of the same custom agent run during one parent session. What is the most important fix?

The dashboard code below was written when only one instance of each agent ran per session.

 event-handler.ts

```
const runs = new Map<string, { status: string; error?: string }>();

session.on("event", (event) => {
  if (event.type === "subagent.started") {
    runs.set(event.data.agentName, { status: "running" });
  }

  if (event.type === "subagent.completed") {
    runs.set(event.data.agentName, { status: "completed" });
  }

  if (event.type === "subagent.failed") {
    runs.set(event.data.agentName, {
      status: "failed",
      error: event.data.error
    });
  }
});
```

- A** Key run state by toolCallId and keep agentName as metadata for display. ✓
- B** Replace the event listener with polling of final pull request status only.
- C** Ignore failed events until all agents emit completed or deselected events.
- D** Store events by agentDisplayName because display names are intended for UI rendering.

✓ **Why the correct answer is right**

The same agent can have multiple execution spans, so toolCallId is the correct correlation key for started, completed, and failed events. agentName remains useful display metadata.

✗ **Why the other options are wrong**

- **B:** Final PR status loses the execution-tree detail needed to diagnose which worker failed.
- **D:** Display names are not execution identifiers and can collide across repeated runs.
- **C:** Ignoring failed events delays incident response and can overwrite the true failure state.

💡 When telemetry includes an explicit correlation identifier, use it; human-readable names are usually labels, not keys.

Q29

Domain C: Memory, state, and execution

multi select

very_hard

A developer has used Copilot CLI for a long refactor involving planning, implementation, and repeated test fixes. After several hours, Copilot starts proposing changes that conflict with an earlier decision to keep the public API stable. Select THREE actions that best manage continuity and drift.

The developer wants to preserve useful work, understand what context may have been summarized, and continue safely if the session is still recoverable.

- A Use ``/context`` to inspect context usage and determine whether compaction likely affected continuity. ✓
- B Delete local session state so Copilot cannot reference any earlier incorrect direction.
- C Increase tool-call verbosity so later context contains every command and file-read result.
- D Re-anchor the session to the original requirement or restart from a reliable checkpoint. ✓
- E Review the session checkpoints to see whether the API-stability decision survived compaction. ✓
- F Continue accepting the proposed edits because automatic compaction preserves every prior detail.

✓ **Why the correct answer is right**

Checkpoints help review what was preserved during compaction, ``/context`` shows context usage, and re-anchoring or restarting from a reliable checkpoint corrects drift without discarding all valid work.

✗ **Why the other options are wrong**

- **C:** More raw tool output consumes context and can worsen drift by retaining low-value details.
- **F:** Compaction summarizes important information but may lose fine-grained details, so review is warranted.
- **B:** Deleting session state removes continuity and makes recovery harder than checkpoint-based correction.

💡 For long-running execution questions, look for deliberate compaction awareness, checkpoint review, and re-anchoring to original requirements.

Q30**Domain A: Agent architecture and SDLC integration**

single choice

hard

An organization owner wants to pilot a new security-remediation custom agent with only the platform team before releasing it to all repositories in the organization. Which repository and path strategy best supports this rollout?

The organization has prepared a `.github-private` repository from the custom agents template. The platform team needs to test and then release the same profile broadly.

- A Place the agent in every target repository under `.github/agents`, then delete copies after testing.
- B Test under `.github-private/.github/agents`, then move the profile to `.github-private/agents` for release. ✓**
- C Create the agent in a public repository so all members can discover and test it easily.
- D Store the agent in `.github-private` root until testing ends, then move it to each repository.

✓ **Why the correct answer is right**

GitHub documents testing organization or enterprise custom agents in the `.github/agents` directory of `.github-private`, then releasing by moving the profile into the root agents directory for broad availability.

✗ **Why the other options are wrong**

- **A:** Copying to every repository creates drift and is not the documented organization-level release pattern.
- **C:** A public repository is inappropriate for an organization-scoped custom agent and may expose internal guidance.
- **D:** The root agents directory is the release location, not the private pilot location.

💡 For organization custom-agent rollout, remember the distinction between test location and release location inside `.github-private`.

custom-agents

organization-rollout

release-process

Q31**Domain A: Prepare agent architecture and SDLC processes**

sequence order

hard

A Copilot cloud agent pull request is ready after implementing a requested change to a service module. The repository requires an approving review and required checks before merge. In which order should the team complete the review workflow?

The developer who initiated the agent task wants the change merged quickly but must preserve the independent review requirement.

Correct order:

1. Inspect the pull request diff, session log link, and generated test evidence.
2. Request agent revisions with a pull request comment if review issues remain.
3. Wait for required checks and an eligible independent approval to complete.
4. Merge the pull request only after required repository protections are satisfied.

✓ **Why the correct answer is right**

Review starts with the diff and trace evidence, then uses PR comments for any agent iteration, then relies on required checks and an eligible independent reviewer, and only then permits merge.

✗ **Why the other options are wrong**

- **B-before-A:** Requesting revisions before inspecting the diff and session evidence risks vague or incorrect feedback.
- **D-before-C:** Merging before checks and independent approval violates the repository protection model.
- **C-before-B:** Waiting for final approval before asking for needed fixes can waste reviewer time and produce stale approvals.

💡 For Copilot PR questions, remember that Copilot output is reviewed like any contribution, and the initiator's approval may not satisfy required approval rules.

pull-request-review

agent-pr-governance

evaluation

Q32

Domain A: Prepare agent architecture and SDLC processes

sequence order

hard

A release team allows an agent to update deployment manifests, but production promotion must stay under human control. In which order should the team design the acceptance path?

The repository uses GitHub Actions deployments and has a production environment with required reviewers.

Correct order:

1. Have the agent create changes in a pull request branch rather than production.
2. Require CODEOWNERS review for deployment manifests and workflow-related files.
3. Merge only after required checks and reviews satisfy branch protections.
4. Let the deployment workflow request production environment approval after merge.

✓ **Why the correct answer is right**

Agent work should enter through a PR branch, sensitive file ownership should be reviewed, branch protections should be satisfied before merge, and production deployment should wait for environment approval.

✗ **Why the other options are wrong**

- **B-before-D:** Requesting production approval before merge bypasses the intended PR-based acceptance sequence.
- **D-before-A:** Branch protection should include the required owner review for sensitive deployment files.
- **A-before-C:** Owner review is triggered by a concrete pull request changing owned files.

💡 Separate code acceptance from deployment approval. PR governance decides whether code can merge; environment protection decides whether deployment can proceed.

deployment-governance

environment-protection

codeowners

Q33**Domain A: Agent architecture and SDLC integration****code or config artifact****very_hard**

A team adds Copilot setup steps so the agent can install project dependencies before working. Copilot sessions still show the default environment and do not run the setup job. What is the best fix?

The file has been merged to the default branch at the expected path. The team wants the smallest configuration change that makes Copilot pick up the setup steps.

 `.github/workflows/copilot-setup-steps.yml`

```
name: Copilot Setup Steps
on:
  workflow_dispatch:
  push:
    paths:
      - .github/workflows/copilot-setup-steps.yml

jobs:
  setup:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
```

- A** Move the workflow to `.github/copilot-setup-steps.yml` so it is outside normal Actions workflows.
- B** Grant `contents: write` permission so Copilot can push dependency cache updates from setup.
- C** Add `pull_request` to the workflow triggers so Copilot can validate the setup file during review.
- D** Rename the job to `copilot-setup-steps` while keeping the file at `.github/workflows/copilot-setup-steps.yml`. ✓

✓ **Why the correct answer is right**

The setup workflow must be located at `.github/workflows/copilot-setup-steps.yml` and contain a single job named `copilot-setup-steps`. The current job name `setup` prevents Copilot from using it.

✗ **Why the other options are wrong**

- **C:** `pull_request` is useful for validation, but the missing required job name is the pickup blocker.
- **A:** The documented location is under `.github/workflows`, not directly under `.github`.

- **B:** The setup job should use least privilege; write permission is unnecessary for installing dependencies.

💡 For setup-step artifacts, check the required file path and required job name before tuning triggers or permissions.

copilot-setup-steps

environment-configuration

configuration-validation

Q34

Domain D: Evaluation, telemetry, error analysis, and tuning

multi select

very_hard

An agent connected to an internal issue-tracking MCP server is asked to link a pull request to the corresponding incident. The trace shows it called the issue search tool with the pull request number instead of the incident ID, then created a link to an unrelated incident. Select TWO actions that best address the root cause.

The MCP server worked correctly when called manually with the incident ID, and the agent had read access to the pull request body where the incident ID appeared.

- A** Classify the failure as tool misuse and refine instructions to extract the incident ID before linking. ✓
- B** Broaden MCP permissions so the agent can inspect all incidents before choosing a link.
- C** Classify the failure as an environment outage because the created link was incorrect.
- D** Disable the MCP server for all agents until the issue-tracking vendor changes the API.
- E** Add a verification step that reads the incident title before committing the pull request link. ✓

✓ **Why the correct answer is right**

The tool call used the wrong parameter even though the needed incident ID was available, so the root cause is tool misuse. Refining instructions and adding a verification step tune tool use without expanding permissions.

✗ **Why the other options are wrong**

- **D:** The MCP server behaved correctly with valid input, so disabling it globally is disproportionate.
- **C:** An incorrect output is not evidence of an environment outage when the trace shows wrong parameters.
- **B:** Broader access increases risk and does not fix the parameter-selection failure.

💡 For tool failures, distinguish bad tool behavior from bad tool invocation. The fix is usually parameter guidance, verification, or least-privilege tool refinement.

tool-misuse

mcp-tools

failure-remediation

Q35**Domain E: Multi-agent orchestration and incident response****single choice****hard**

A platform group is deciding how to redesign incident rollback automation. The decision affects reliability, cost, and compliance, and the team wants multiple independent proposals before choosing. Which orchestration pattern best fits this decision?

The agents should not edit code yet. The desired output is a set of competing designs with trade-offs for a human architecture review board.

- A Use a pipeline where each agent rewrites the previous proposal into its preferred style.
- B Use debate or consensus so agents independently propose designs for human selection. ✓**
- C Use parallel fan-out where agents implement separate modules before the design is approved.
- D Use a single implementation agent with broad tools to converge quickly on one design.

✓ **Why the correct answer is right**

Debate or consensus fits situations where multiple agents independently produce alternatives and a human or orchestrator selects the best option based on trade-offs.

✗ **Why the other options are wrong**

- **A:** A pipeline would make each proposal depend on the previous one, reducing independence.
- **C:** Implementation before design approval creates unnecessary change risk for a governance decision.
- **D:** A single broad agent does not provide the independent viewpoints requested by the review board.

💡 When the stem asks for competing viewpoints before action, choose debate or consensus rather than execution-oriented patterns.

debate-consensus

architectural-decision

human-review

Q36

Domain B: MCP/tool use and environment permissions

multi select

hard

A team adds the following setup workflow so Copilot cloud agent can install dependencies before editing a service. The agent still starts without dependencies installed. Select TWO changes that best align the workflow with Copilot setup-step requirements and least privilege.

The workflow file is intended to run before Copilot starts work and is stored at `.github/workflows/copilot-setup-steps.yml`.

 `.github/workflows/copilot-setup-steps.yml`

```
name: Copilot Setup Steps
on:
  workflow_dispatch:
  pull_request:
    paths:
      - .github/workflows/copilot-setup-steps.yml
jobs:
  setup:
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
```

- A** Rename the job from setup to copilot-setup-steps so Copilot can pick it up. ✓
- B** Add a second job named validate-dependencies that runs after the setup job completes.
- C** Move dependency installation into a separate deployment workflow triggered after pull_request.
- D** Add issues: write permission so Copilot can report dependency installation failures.
- E** Change contents: write to contents: read because checkout only needs read access. ✓

✓ **Why the correct answer is right**

The setup workflow must contain a single job named copilot-setup-steps to be used by Copilot cloud agent. Since the setup steps only check out code and install dependencies, contents: read is the least-privilege permission.

✗ **Why the other options are wrong**

- **B:** Additional jobs do not satisfy the required Copilot setup job name and may be ignored for this purpose.
- **D:** Issue write permission is unrelated to dependency installation and expands the token permissions unnecessarily.
- **C:** A separate deployment workflow runs at the wrong point in the lifecycle and does not configure the agent environment before work starts.

💡 For copilot-setup-steps.yml items, look first for the exact job name and then for whether permissions match the setup steps, not the agent's later work.

copilot-setup-steps

environment-permissions

least-privilege

Q37

Domain A: Agent architecture and SDLC integration

single choice

very_hard

A monorepo team asks Copilot cloud agent to update an API client in Repository A and simultaneously patch the matching service implementation in Repository B. The agent creates changes only in Repository A. What should the architect recommend?

Both repositories are hosted on GitHub. The organization wants traceability and does not want an agent run to make uncontrolled cross-repository changes.

- A Use a broader custom agent prompt that names both repositories and asks Copilot to infer coordination.
- B Grant the agent organization-owner permissions so it can push branches across both repositories.
- C Split the work into coordinated repository-scoped tasks with separate branches, PRs, checks, and reviews. ✓
- D Disable branch protection temporarily so the two repository changes can be merged as one operation.

✓ **Why the correct answer is right**

Copilot cloud agent can make changes only in the repository specified when the task starts and can work on one branch at a time. Coordinated repository-scoped tasks preserve traceability and review controls.

✗ **Why the other options are wrong**

- **B:** Over-permissioning does not change the documented single-repository task boundary and increases risk.
- **A:** Prompting cannot safely override repository execution boundaries.
- **D:** Disabling protection weakens governance and does not solve cross-repository traceability.

💡 For scope-limit questions, do not choose answers that bypass governance; decompose work into bounded, auditable tasks.

agent-scope

repository-boundaries

sdhc-design

Q38**Domain B: MCP/tool use and environment permissions****single choice****hard**

A Copilot cloud agent pull request includes a warning that a command attempted to reach `downloads.vendor.example` and was blocked. The dependency is legitimate and required by tests, but the security team wants to preserve exfiltration controls. What should the repository administrator do?

The organization-level firewall policy lets repositories manage custom allowlists, but disabling the firewall would require a security exception.

- A** Move the dependency download into an MCP server because the firewall blocks Bash commands.
- B** Store the vendor URL as an Agents secret so the firewall treats it as trusted.
- C** Disable the firewall for the repository until the dependency installation process is stable.
- D** Add the specific vendor host to the custom allowlist and keep the firewall enabled. ✓

✓ **Why the correct answer is right**

Copilot cloud agent's firewall limits internet access and surfaces blocked addresses. If the domain is required, the safer response is to allowlist the specific host while keeping the firewall enabled.

✗ **Why the other options are wrong**

- **C:** Disabling the firewall removes a broad exfiltration control for a problem that has a targeted allowlist solution.
- **A:** MCP servers are not the right workaround for dependency downloads and are not covered by the same firewall limitation.
- **B:** Secrets and variables provide environment values; they do not automatically grant network access through the firewall.

💡 Firewall questions often include a tempting disable option. Prefer precise allowlisting when the required endpoint is known and legitimate.

agent-firewall

network-allowlist

environment-constraints

Q39**Domain E: Multi-agent orchestration and incident response**

single choice

medium

A service team is responding to a checkout incident. One agent must inspect logs and historical PRs, another must make a minimal code fix, and a third must validate tests after the fix. Which orchestration approach best fits the situation?

The team wants the main agent to preserve context, avoid giving write tools to the research task, and integrate the final result before requesting review.

- A** Use an orchestrator-worker flow that delegates scoped subtasks to specialized agents, then integrates their outputs. ✓
- B** Ask each agent to produce a full independent fix and let CI determine the final choice.
- C** Use a single broad agent with all tools so it can switch tasks without handoff overhead.
- D** Run all three agents against the same branch and merge whichever output finishes first.

✓ **Why the correct answer is right**

An orchestrator-worker flow matches the need for decomposition, scoped tools, isolated sub-agent execution, event streaming, and integration by the parent session.

✗ **Why the other options are wrong**

- **C:** A broad agent increases context clutter and tool exposure instead of using scoped specialists.
- **D:** Sharing one branch increases the chance of overlapping edits and makes attribution harder.
- **B:** Independent full fixes waste effort and defer architectural judgment to CI, which cannot evaluate intent.

💡 For GH-600 orchestration items, map the dependency structure first: distinct specialized subtasks with final integration usually indicate orchestrator-worker.

orchestrator-worker

agent-isolation

incident-response

Q40**Domain A: Prepare agent architecture and SDLC processes**

code or config artifact

very_hard

A platform team is preparing an organization-level custom agent for high-risk database migration work. The security review requires the agent to produce an inspectable design artifact before any repository edits occur, and a human must approve that plan before implementation begins. Which revision best satisfies the requirement while preserving delivery velocity?

The organization stores shared custom agents in a private `.github-private` repository. The draft agent profile is intended to be released as `agents/migration-planner.agent.md`.

 `agents/migration-planner.agent.md`

```
---
name: migration-planner
description: Designs database migration work and opens implementation pull requests
tools: ["read", "search", "edit", "runCommands"]
target: github-copilot
---
```

When assigned an issue:

1. Inspect the schema and application code.
2. Decide the migration approach.
3. Apply required edits, run tests, create a pull request, and mark it ready for review.
4. If CI passes, summarize the outcome and ask for merge.

- A** Split this into a planning-only custom agent with read/search tools, structured plan output, and a human approval gate before invoking an implementation agent. ✓
- B** Move it to each repository's `.github/agents` folder and rely on repository visibility to limit use.
- C** Keep one agent, but add CODEOWNERS and required checks so unsafe edits are blocked during pull request review.
- D** Allow edit tools, but instruct the agent to self-review and request human review only when tests fail.

✓ **Why the correct answer is right**

The requirement is to separate planning from execution, produce a structured and reviewable plan, and prevent action until a human approves. A planning-only agent with read/search tools limits autonomy at the design stage, creates an inspectable artifact, and still preserves velocity by allowing an implementation agent after approval.

✗ **Why the other options are wrong**

- **C:** CODEOWNERS and checks help govern pull requests, but they act after edits occur. They do not enforce the required plan-before-action boundary.

- **D:** Self-review and test-gated review leave execution in the agent's hands before approval. This keeps the risky edit capability inside the planning phase.
- **B:** Repository-level placement can narrow availability, but it does not solve the core issue: the agent still mixes planning, editing, and execution.

💡 For GH-600 architecture questions, identify where the risk occurs in the lifecycle. If the requirement says approval before action, choose the design that removes action tools from the planning phase rather than relying only on downstream PR controls.

planning-execution-boundaries

human-in-the-loop

custom-agent-governance

structured-artifacts

Q41

Domain E: Multi-agent orchestration and incident response

single choice

hard

A compliance reviewer asks how the team will prove which agent introduced a risky dependency during a multi-agent remediation effort. What evidence design best supports the investigation?

The repository uses Copilot cloud agent PRs, and the team needs evidence that remains useful after the incident is closed.

- A Store a screenshot of the agents dashboard at the end of each workday.
- B Require session logs, commit links to those logs, and PR handoff notes for each agent. ✓**
- C Rely on the final merged diff because intermediate agent decisions are rarely needed.
- D Ask agents to summarize their own quality in a final comment after merging.

✓ **Why the correct answer is right**

Copilot session logs show approach and tool use, and Copilot-authored commits include links to session logs. PR handoff notes add decision context for later review.

✗ **Why the other options are wrong**

- **C:** The final diff does not explain which agent made a decision or why it used a tool.
- **D:** A post-merge self-summary is weaker than source evidence collected during execution.
- **A:** Screenshots are incomplete, hard to correlate to commits, and poor substitutes for logs.

💡 Auditability answers should tie operational signals to durable development artifacts such as commits, PRs, and logs.

auditability

session-logs

handoff-documentation

Q42

Domain A: Prepare agent architecture and SDLC processes

matching magnet

hard

A team is rewriting its issue templates so tasks assigned to agents are easier to evaluate. Match each template field to the role it should play in a structured agent task.

The team wants reviewers to quickly determine whether Copilot cloud agent had enough context and whether the resulting pull request is acceptable.

Correct matches:

Branch name, file paths, issue description, and affected service. → Inputs

Expected commit diff, PR description, and test evidence. → Outputs

Required checks passing and no new secrets or policy violations. → Success criteria

Known exclusions and changes the agent must not attempt. → Out-of-scope boundaries

Choices: Out-of-scope boundaries, Inputs, Outputs, Success criteria, Long-term memory facts, Model selection preferences

✓ **Why the correct answer is right**

A well-defined agent task separates the context the agent receives, the artifacts it should produce, the objective acceptance criteria, and the boundaries that limit scope creep.

💡 When a stem mentions issue templates for agents, classify fields by whether they tell the agent where to start, what to produce, how to pass, or what to avoid.

task-definition

inputs-outputs-success

sdic-contract

Q43

Domain B: MCP/tool use and environment permissions

code or config artifact

medium

A data-science repository adds the following MCP configuration so Copilot cloud agent can consult experiment metadata. Copilot reports that the configuration is not usable. What is the most appropriate fix?

The metadata server exposes a `search_experiments` tool and also publishes prompt templates for human analysts.

 Repository MCP configuration

```
{
  "mcpServers": {
    "experiments": {
      "type": "http",
      "url": "https://ml-metadata.contoso.example/mcp",
      "resources": ["experiments://latest"],
      "prompts": ["compare-runs"]
    }
  }
}
```

- A** Replace resources and prompts with a tools allowlist such as `["search_experiments"]`. ✓
- B** Add a prompt that instructs Copilot to call `compare-runs` before changing files.
- C** Move resources and prompts under headers so they are sent with each request.
- D** Convert the resources array to a string so Copilot can load the latest experiments resource.

✓ **Why the correct answer is right**

Copilot cloud agent supports tools provided by MCP servers and does not support MCP resources or prompts. The configuration should expose only the required server tool through the tools list.

✗ **Why the other options are wrong**

- **D:** Changing the JSON type of resources does not make MCP resources supported by Copilot cloud agent.
- **B:** Prompt templates are not supported as MCP capabilities for Copilot cloud agent in this configuration.
- **C:** Headers are for request metadata such as authentication values, not for declaring MCP resources or prompts.

💡 If an MCP artifact contains resources or prompts, the exam is usually testing whether you know cloud agent consumes MCP tools only.

mcp-artifact-evaluation

cloud-agent-support

tool-allowlist

Q44**Domain F: Guardrails, safety, accountability, and governance**

single choice

medium

A release engineering team wants Copilot cloud agent to update deployment manifests and trigger a staging deployment for review apps. The deployment can modify shared test data but cannot affect production customers. What autonomy level and guardrail best balances velocity and control?

The repository already requires CI checks before merge, and the team wants to avoid approvals for simple code edits that do not deploy anything.

- A** Let Copilot deploy to staging automatically, then rely on audit logs to identify problematic changes later.
- B** Disable Copilot cloud agent for the repository whenever deployment manifests are part of the requested change.
- C** Require a security manager approval before Copilot can create any commits or open a draft pull request.
- D** Allow code changes autonomously, but require a protected staging environment approval ✓

✓ **Why the correct answer is right**

Staging deployment is a higher-risk action than editing code, so the efficient guardrail is to keep artifact creation autonomous while placing the human checkpoint at the protected environment boundary.

✗ **Why the other options are wrong**

- **C:** Approving every commit or draft PR slows low- and medium-risk work that can be controlled by reviews and required checks.
- **A:** Monitoring after deployment is weaker than a pre-deployment approval for an action that can affect shared data.
- **B:** Disabling the agent removes useful delivery velocity instead of scoping the human checkpoint to the risky action.

💡 For GH-600 guardrail questions, place approvals at the risk boundary. Code generation usually needs artifact review; deployments need environment protection.

autonomy-levels

human-in-the-loop

environment-protection

Q45**Domain F: Guardrails, safety, accountability, and governance**

multi select

very_hard

A repository needs Copilot cloud agent to install private build tools before editing code. The security team requires least privilege and does not want the agent to access deployment credentials. Select TWO configuration decisions that satisfy the requirement.

The current proposal grants broad GITHUB_TOKEN permissions in copilot-setup-steps.yml and reuses GitHub Actions secrets used by release workflows.

- A** Provide only required build credentials through Copilot agent secrets or variables, separate from release credentials. ✓
- B** Grant actions: write and contents: write so Copilot can repair failed setup workflows during the same run.
- C** Move deployment secrets into repository-level Actions secrets so Copilot can read them when setup needs authentication.
- D** Use a long-lived personal access token with organization-wide write access to avoid repeated permission changes.
- E** Disable the setup workflow on pull_request events to prevent validation from exposing dependency installation failures.
- F** Set only the minimum GITHUB_TOKEN permissions needed by setup steps, such as contents: read for checkout. ✓

✓ **Why the correct answer is right**

The setup workflow should use the lowest token permissions required for setup, and secrets needed by the agent should be provided through the Copilot agent secrets or variables mechanism rather than reusing unrelated release credentials.

✗ **Why the other options are wrong**

- **B:** Broad write permissions increase blast radius and are not needed for dependency installation or checkout.
- **C:** Copilot cloud agent does not access GitHub Actions secrets during runtime or environment setup steps, and deployment credentials should remain outside the agent boundary.
- **E:** Disabling validation hides setup failures and does not improve least-privilege design.
- **D:** An organization-wide write token is over-scoped for installing build tools and increases credential risk.

💡 In multi-select least-privilege questions, choose both sides of the boundary: minimal token permissions and the correct secret mechanism.

least-privilege

setup-steps

secrets-boundary

Q46

Domain B: MCP/tool use and environment permissions

multi select

hard

A dependency-update agent creates a branch, edits four files, and then encounters repeated 429 responses from a package registry during validation. The team requires safe execution paths and auditable recovery. Select TWO actions the workflow should take.

The issue is likely transient, but the agent has already made partial changes that should not be silently abandoned.

- A** Retry the registry calls with bounded exponential backoff and record each attempt. ✓
- B** Merge the branch after unit tests pass locally and rerun registry validation later.
- C** Increase the agent's package-registry permissions so rate limiting is less likely.
- D** Escalate with a comment and roll back or preserve the branch as a draft for inspection. ✓
- E** Continue with the pull request and let maintainers discover validation gaps during review.

✓ **Why the correct answer is right**

Transient failures should be retried with bounded backoff and logged. If retries fail or confidence is low, the agent should surface the failure to humans and either roll back partial work or preserve it in an inspectable draft state.

✗ **Why the other options are wrong**

- **E:** Proceeding without surfacing the validation failure hides an important tool error from reviewers.
- **C:** Permission expansion does not address a transient 429 safely and may violate least privilege.
- **B:** Passing local tests does not substitute for the failed registry validation required by the workflow.

💡 For safe-execution questions, combine bounded automation with human-visible escalation; avoid options that hide partial failures or expand permissions reactively.

error-handling

rollback

escalation

Q47**Domain A: Agent architecture and SDLC integration****single choice****medium**

A product team wants Copilot cloud agent to handle small accessibility fixes across a React repository. The first pilot issue says only: "Improve accessibility on the settings page." The agent produced a broad refactor that passed tests but changed unrelated navigation behavior. What should the team do before assigning similar issues to the agent?

The team wants to keep the agent in the SDLC, but reviewers need predictable scope and reviewable outputs.

- A** Assign only senior developers as reviewers and keep the issue format unchanged for velocity.
- B** Rewrite issues with explicit inputs, affected paths, expected diff artifacts, tests to run, and acceptance criteria. ✓
- C** Require the agent to create fewer commits, so reviewers can inspect a smaller commit history.
- D** Increase the agent model capability and allow it to infer missing acceptance criteria from repository history.

✓ **Why the correct answer is right**

The scenario shows an underspecified task. Domain A expects agent tasks to define inputs, outputs, and success criteria so the agent can produce bounded, inspectable work inside normal GitHub review workflows.

✗ **Why the other options are wrong**

- **D:** A stronger model does not replace explicit task boundaries; it can still infer scope incorrectly.
- **A:** Reviewer expertise helps catch problems, but it does not prevent the agent from starting with vague goals.
- **C:** Commit count affects review ergonomics, not whether the task is scoped to the right work.

💡 When a GH-600 question describes an agent doing too much, look for the answer that improves task definition before execution.

sdlc-integration

task-boundaries

success-criteria

Q48**Domain F: Guardrails, safety, accountability, and governance****code or config artifact****hard**

A platform team audits repository guardrails before enabling Copilot cloud agent in a service that uses custom instructions, custom agents, MCP, and setup steps. Based on the CODEOWNERS file, which change best protects agent configuration from unreviewed modification?

The repository already requires CODEOWNERS review on pull requests that modify owned files.

 **.github/CODEOWNERS**

```
# Existing ownership rules
.github/workflows/* @octo-org/devops
.github/dependabot.yml @octo-org/security
mcp.json @octo-org/platform
src/payments/** @octo-org/payments-maintainers
```

- A** Replace all entries with a single * rule owned by security to centralize every repository review.
- B** Keep the file unchanged because .github/workflows/* already covers every Copilot and MCP configuration path.
- C** Remove workflow ownership and depend on required status checks to detect unsafe agent configuration edits.
- D** Add owners for Copilot instructions, path instructions, custom agents, MCP settings, and copilot setup steps. ✓

✓ **Why the correct answer is right**

Guardrails guidance recommends protecting important Copilot and MCP configuration files with CODEOWNERS and requiring Code Owner review. The current file covers workflows broadly and one MCP path, but it misses Copilot instruction and custom-agent files.

✗ **Why the other options are wrong**

- **B:** The workflow glob does not cover repository instructions, path-specific instructions, or custom agent profiles.
- **A:** A broad security-owner rule creates unnecessary review load and may slow unrelated engineering changes.
- **C:** Status checks validate behavior after a change is proposed; they do not ensure the right humans review guardrail configuration changes.

💡 When a question shows governance configuration, look for the smallest targeted ownership rule that protects control-plane files without centralizing all reviews.

codeowners

configuration-protection

rulesets

Q49**Domain A: Agent architecture and SDLC integration****code or config artifact****hard**

A platform team is creating a repository-level custom agent for documentation-only changes. A reviewer notices the agent can edit source files and run shell commands even though its prompt asks it not to. Which change best aligns the configuration with the intended boundary?

The team wants the custom agent to inspect repository files and update Markdown documentation, but not execute commands or invoke every configured MCP tool.

 `.github/agents/docs-maintainer.agent.md`

```
---
name: docs-maintainer
description: Updates README files, ADRs, and Markdown guides for the service
---
You are a documentation specialist. Update only Markdown documentation.
Do not modify application code, workflow files, or package manifests.
```

- A** Set user-invokable to false and rely on automatic routing to select the agent only for docs tasks.
- B** Move the agent into the organization `.github-private` repository so fewer repository users edit it.
- C** Add a tools list limited to read, search, and edit, and keep the prompt scoped to Markdown. ✓
- D** Add more prompt text explaining that application files are confidential and should be avoided.

✓ **Why the correct answer is right**

If the tools property is omitted, custom agents can access all available tools by default. Explicitly listing only needed tools applies a stronger boundary than prompt-only guidance.

✗ **Why the other options are wrong**

- **D:** Prompt guidance helps behavior, but it does not reduce available tools.
- **B:** Changing storage location can affect availability, but it does not narrow tool access.
- **A:** Invocation settings affect selection, not what the agent can do after it runs.

💡 When a question shows prompt-only safety, look for configuration controls that narrow tools or execution paths.

custom-agents

least-privilege

configuration-review

Q50**Domain D: Evaluation, telemetry, error analysis, and tuning**

single choice

hard

A repository owner finds that Copilot cloud agent repeatedly proposes changes using a legacy database helper that was removed during a migration. The session log shows Copilot citing a stored repository fact about that helper, and reviewers confirm the fact is no longer accurate. What should the owner do first to tune future agent behavior?

The team wants to reduce repeated rework without disabling Copilot for the repository.

- A** Ask developers to mention the new database helper in every future Copilot prompt.
- B** Disable all custom instructions until Copilot relearns the codebase from new pull requests.
- C** Add a branch protection rule requiring two approvals for any database-related pull request.
- D** Review Copilot Memory for the repository and delete the misleading stored fact. ✓

✓ **Why the correct answer is right**

Repository owners can view and delete stored Copilot Memory entries that are inappropriate, misleading, or incorrect. Removing the stale fact tunes the context source causing repeated wrong proposals.

✗ **Why the other options are wrong**

- **B:** Custom instructions are not identified as the faulty context source, so disabling them is overbroad.
- **C:** Additional approvals may catch the issue but do not remove the stale context causing repeated failures.
- **A:** Repeating corrective context in every prompt increases manual burden instead of fixing the repository-level memory issue.

💡 When repeated errors trace to stored context, tune the memory source rather than adding review friction or relying on every prompt to compensate.

memory-tuning

context-drift

copilot-memory

Q51**Domain E: Multi-agent orchestration and incident response****code or config artifact****hard**

A team adds a cleanup sub-agent to an SDK-based orchestrator used during incident response. Reviewers worry that routine incident prompts could accidentally invoke the cleanup agent and delete useful diagnostic files. Which change best addresses the risk?

The cleanup agent should be available only when a human explicitly requests it, and it should not inherit unnecessary tools.

 **orchestrator.ts**

```
const session = await client.createSession({
  model: "gpt-4.1",
  customAgents: [
    {
      name: "cleanup",
      displayName: "Cleanup Agent",
      description: "Deletes obsolete files, removes dead code, and prunes unused assets",
      tools: null,
      prompt: "Clean up the repository after incidents. Remove files that appear unused."
    }
  ],
  onPermissionRequest: async () => ({ kind: "approved" })
});
```

- A** Set infer to false and replace tools null with an explicit minimal tool list. ✓
- B** Keep tools null and add stronger natural-language warnings in the cleanup prompt.
- C** Remove the description so the runtime has less information for intent matching.
- D** Preselect the cleanup agent at session creation so invocation is predictable.

✓ **Why the correct answer is right**

In the SDK, infer false prevents automatic selection, and an explicit tools array avoids inheriting all session tools when tools is null or omitted.

✗ **Why the other options are wrong**

- **B:** Prompt warnings do not remove automatic selection or broad inherited tool access.
- **C:** A missing description can reduce routing quality without enforcing explicit invocation or tool scope.
- **D:** Preselecting the cleanup agent makes it active at session start, increasing the accidental-use risk.

💡 When a sub-agent is sensitive, prefer configuration controls such as explicit invocation and scoped tools over prompt-only safeguards.

Q52

Domain A: Prepare agent architecture and SDLC processes

sequence order

hard

An organization owner is preparing organization-level custom agents so multiple repositories can use a standardized refactoring agent. In which order should the owner set up and release the agent repository?

The owner wants to follow GitHub's recommended organization-level custom agent model and avoid publishing an untested agent broadly.

Correct order:

1. Create the custom agent repository from GitHub's template under the organization.
2. Name the repository `.github-private` and add organization-specific authoring guidance.
3. Choose internal or private visibility based on who should read and use agents.
4. Place the tested organization agent profile in the root agents directory for release.

✓ **Why the correct answer is right**

GitHub directs organization owners to create the custom agents repository from the template, name it `.github-private`, choose internal or private visibility, and use the organization-level agents directory for agent profiles.

✗ **Why the other options are wrong**

- **A-before-B:** Visibility is selected as part of creating the repository, after choosing the organization and repository purpose.
- **D-before-B:** An organization-level agent profile cannot be released until the `.github-private` repository exists.
- **D-before-A:** Releasing an agent before setting repository visibility can expose it to the wrong audience.

💡 For custom-agent setup sequences, distinguish repository preparation from agent profile placement. Organization-level agents use `.github-private` and the root agents directory.

Q53

Domain A: Agent architecture and SDLC integration

multi select

very_hard

A Copilot-created pull request modifies application code and also edits `.github/workflows/deploy.yml`. The repository has required checks, but checks have not started. Select TWO actions that preserve delivery speed while maintaining the intended security checkpoint.

The team needs CI results before merge, but wants to avoid running privileged workflows on agent changes without inspection.

- A** Inspect the workflow diff, then have a write-access user click Approve and run workflows if acceptable. ✓
- B** Keep required checks and human review before merge, especially for changes under `.github/workflows`. ✓
- C** Ask Copilot to approve and run the workflows after it explains the workflow file changes.
- D** Disable workflow approval for Copilot pull requests so required checks start immediately.
- E** Merge the pull request with pending checks because Copilot commits are signed and verified.

✓ **Why the correct answer is right**

By default, Actions workflows do not run automatically when Copilot pushes changes. A write-access user should inspect the changes, especially workflow changes, and explicitly approve running workflows; required checks and review remain merge gates.

✗ **Why the other options are wrong**

- **D:** Automatic workflow execution may be configurable, but it weakens the checkpoint described in the scenario.
- **E:** Signed commits improve traceability; they do not replace required checks or review.
- **C:** Copilot cannot supply the human authorization needed for privileged workflow execution.

💡 When Actions are paused on Copilot PRs, the exam usually wants inspection plus explicit human approval, not bypassing checks.

control-plane

actions-workflows

human-intervention

Q54

Domain B: MCP/tool use and environment permissions

multi select

very_hard

A release-engineering agent in repo apps/api needs read-only GitHub context from repos apps/web and platform/contracts to prepare compatibility notes. The default GitHub MCP access only covers the source repository. Select THREE actions that meet the requirement with the narrowest practical access.

The agent must not create issues, update files, or access unrelated repositories through the GitHub MCP server.

- A Remove all tool restrictions and rely on the prompt to request read-only behavior.
- B Create a fine-grained PAT limited to read access on apps/web and platform/contracts. ✓
- C Set github/* in the custom agent tools list to simplify GitHub MCP selection.
- D Store the PAT as an Agents secret named `COPILLOT_MCP_GITHUB_PERSONAL_ACCESS_TOKEN`. ✓
- E Use GitHub MCP toolsets and tool filtering to expose only required read operations. ✓
- F Use an organization classic PAT with repo scope so future agents can reuse it.

✓ **Why the correct answer is right**

For context outside the current repository, GitHub documents using an appropriately scoped PAT, preferably fine-grained. The credential must be stored as a `COPILLOT_MCP_Agents` secret, and toolsets plus tool filtering should constrain which GitHub MCP operations are available.

✗ **Why the other options are wrong**

- **F:** A classic PAT with broad repo scope grants more repository access than the two named repositories require.
- **C:** `github/*` enables all available GitHub MCP tools rather than limiting to required read operations.
- **A:** Prompt instructions do not enforce permission boundaries for autonomously callable MCP tools.

💡 Cross-repository context is a scope-expansion scenario. The right answer expands only the data boundary requested and still filters tools.

github-mcp

cross-repository-context

toolsets

Q55

Domain B: MCP/tool use and environment permissions

code or config artifact

hard

A repository administrator adds the following MCP configuration for Copilot cloud agent to read deployment incidents. The configuration fails validation and would be over-permissive even if it loaded. Which remediation best addresses both issues?

The remote service supports static bearer-token authentication and exposes separate read and write tools.

 Repository MCP configuration

```
{
  "mcpServers": {
    "incidents": {
      "type": "http",
      "url": "https://incidents.contoso.example/mcp",
      "tools": ["*"],
      "headers": {
        "Authorization": "Bearer $INCIDENTS_TOKEN"
      },
      "oauth": {
        "clientId": "copilot-agent"
      }
    }
  }
}
```

- A Change the server type to local, keep OAuth, and run the server through npx.
- B Remove OAuth, reference \$COPILOT_MCP_INCIDENTS_TOKEN, and list only required read tools. ✓**
- C Move the token into a repository Actions secret and reference it as \$INCIDENTS_TOKEN.
- D Keep tools as ["*"], remove the header, and rely on the server's default anonymous read access.

 Why the correct answer is right

Remote MCP configuration supports url and optional headers, but Copilot cloud agent does not support remote OAuth flows. Secret references for MCP must use Agents secrets or variables prefixed with COPILOT_MCP_, and the tools list should allowlist only required read tools.

 Why the other options are wrong

- **D:** Anonymous access does not solve least privilege or authentication and keeps every exposed MCP tool enabled.
- **C:** Actions secrets are not passed to Copilot cloud agent MCP configuration, and the name lacks the required prefix.
- **A:** Changing to a local server adds unnecessary runtime complexity and keeps the unsupported OAuth configuration.

💡 For artifact questions, separate syntax validity from security posture; the best answer usually fixes both rather than only making the file parse.

mcp-json

remote-mcp

secret-prefix

Q56

Domain B: MCP/tool use and environment permissions

code or config artifact

hard

A security team reviews the following custom agent profile before enabling it for dependency advisory triage. The agent should read advisories and repository files, then create a Markdown plan without running commands or changing application code. What is the best change?

The repository has a vulnerability-intelligence MCP server configured with both read and mutation tools.

 `.github/agents/advisory-planner.md`

```
---
name: advisory-planner
description: Reviews dependency advisories and drafts remediation plans
mcp-servers:
  vuln-intel:
    type: http
    url: https://vuln-intel.contoso.example/mcp
    tools: ["*"]
    headers:
      Authorization: Bearer $COPILOT_MCP_VULN_INTEL_TOKEN
---
```

Create a remediation plan in docs/security/remediation-plan.md.

- A Keep the profile unchanged because the instructions restrict the agent to Markdown output.
- B Add execute so the agent can verify package-manager behavior before drafting the plan.
- C Remove the MCP server and ask developers to paste advisory data into the issue.
- D Set tools to ["read", "search", "edit", "vuln-intel/advisory-read"] and restrict MCP tools. ✓**

✓ Why the correct answer is right

The profile currently omits the top-level tools property, which enables all available tools, and the MCP server enables every server tool. Explicitly listing read, search, edit, and the one required MCP read tool matches the task while avoiding command execution and mutation tools.

✗ Why the other options are wrong

- **A:** Natural-language instructions do not remove tool capabilities that the agent is configured to use.
- **B:** Shell execution is not required for drafting a remediation plan and expands the allowed action surface.
- **C:** Removing MCP access reduces useful context and shifts manual work to developers instead of scoping the tool correctly.

 In GH-600 config artifacts, instructions and tool permissions are separate controls; do not treat prompt wording as a substitute for tool scoping.

Q57

Domain E: Multi-agent orchestration and incident response

single choice

medium

A team automates release-note generation after incidents. A triage agent must first identify merged incident PRs, a summarizer must convert them into user-facing notes, and a reviewer agent must check tone and policy wording. Which orchestration pattern should the team choose?

Each step depends on the previous step's output, and running later agents before earlier outputs are complete causes inaccurate notes.

- A Use parallel fan-out so each agent produces release notes independently from the repository.
- B Use orchestrator-worker so the parent agent assigns every step at the same time.
- C Use a pipeline so each agent transforms the prior agent's output in sequence. ✓
- D Use debate consensus so all agents argue for the most complete final summary.

✓ **Why the correct answer is right**

A pipeline fits sequential transformations where agent N's output becomes agent N+1's input, such as triage to summary to policy review.

✗ **Why the other options are wrong**

- **A:** Parallel fan-out is better for independent subtasks, not ordered dependent transformations.
- **D:** Debate consensus is useful for comparing alternatives, not enforcing a fixed processing chain.
- **B:** Concurrent delegation risks later agents working before required intermediate artifacts exist.

💡 When the stem emphasizes ordered dependencies, choose pipeline rather than fan-out or debate.

Q58

Domain A: Prepare agent architecture and SDLC processes

matching magnet

very_hard

A staff engineer reviews four failed agent tasks and wants to classify the primary design problem in each one. Match each failure report to the most likely anti-pattern.

The goal is to choose mitigations for future runs, not to debug the individual code changes.

Correct matches:

The PR says all checks were acceptable, but the session log shows a linter command failed. → Silent failure

The agent reused a customer-specific workaround from a prior unrelated issue. → Context leakage

The agent made schema, API, and deployment changes from one vague task. → Unbounded autonomy

Reviewers received paragraphs of rationale but no file list, plan, or test summary. → No structured output

Choices: Context leakage, Silent failure, No structured output, Unbounded autonomy, Insufficient model capability, Too many required reviewers

✓ Why the correct answer is right

Hidden failed commands indicate silent failure, unrelated prior context indicates leakage, broad unsupervised scope indicates unbounded autonomy, and prose without artifacts indicates missing structured output.

💡 Failure-classification items are easiest if you name the risk first: hidden errors, leaked context, excessive scope, or unreviewable output.

failure-analysis

anti-patterns

control-design

Q59**Domain E: Multi-agent orchestration and incident response**

single choice

hard

Two Copilot cloud agent sessions are repairing the same incident from different issues. One PR changes authentication middleware to fail closed, while another PR changes a caller to retry unauthenticated requests. Both PRs pass their local tests. What should the incident orchestrator do next?

The incident commander needs a defensible decision trail and cannot merge changes that encode contradictory security behavior.

- A Close both PRs and restart the entire incident fix as one new agent session.
- B Pause merging, request human review of both designs, choose one, and document the rejection. ✓**
- C Merge the PR with the larger test suite because passing tests indicate the safer behavior.
- D Ask both agents to rebase repeatedly until GitHub reports no merge conflicts.

✓ **Why the correct answer is right**

Contradictory outputs are not just textual conflicts; they require human judgment, an explicit choice, and an auditable rationale before work continues.

✗ **Why the other options are wrong**

- **C:** More tests do not resolve conflicting security intent or prove the architecture is correct.
- **D:** A clean rebase can hide semantic disagreement while leaving incompatible behavior in the system.
- **A:** Restarting discards useful work and evidence that can support a human-in-the-loop decision.

💡 Semantic conflicts require judgment. Do not confuse passing CI or clean merges with resolved design disagreement.

agent-conflict

human-in-the-loop

contradictory-outputs

Q60

Domain B: MCP/tool use and environment permissions

code or config artifact

very_hard

An SDK-based orchestrator should not run shell commands itself, but it must delegate test execution to a specialized sub-agent. Which change best implements that boundary without blocking the test sub-agent?

The current configuration uses a session-wide exclusion and the test-runner agent cannot execute npm test.

 Copilot SDK session configuration

```
const session = await client.createSession({
  availableTools: ["read", "edit", "execute", "agent"],
  excludedTools: ["execute"],
  customAgents: [
    {
      name: "test-runner",
      description: "Runs test commands and summarizes failures",
      tools: ["read", "execute"],
      prompt: "Run only project test commands and report failures."
    }
  ]
});
```

- A Remove execute from availableTools and let test-runner request it during execution.
- B Keep excludedTools unchanged and add execute twice in the test-runner tools array.
- C Move execute restriction to defaultAgent.excludedTools and keep execute session-available. ✓
- D Give the main agent execute access and depend on prompt instructions not to use it.

✓ **Why the correct answer is right**

In Copilot SDK, session-level availableTools and excludedTools apply globally before default-agent restrictions. defaultAgent.excludedTools can hide a tool from the main agent while leaving it available to sub-agents when the tool remains session-available.

✗ **Why the other options are wrong**

- **A:** Removing execute from availableTools prevents every agent from using it, including the test-runner sub-agent.
- **B:** A session-level excluded tool remains unavailable regardless of how often the sub-agent lists it.
- **D:** Granting execute to the main agent violates the intended separation between orchestration and execution.

💡 For SDK filtering questions, determine whether a filter is global or main-agent-only; global exclusions override sub-agent tool lists.

Q61

Domain D: Evaluation, telemetry, error analysis, and tuning

single choice

medium

A product infrastructure group asks Copilot cloud agent to modernize deprecated logging calls across several services. The goal is to reduce human cleanup while avoiding behavior changes. Which success criteria set is most appropriate?

The work is repetitive but touches production services, so the team wants evaluation signals that detect both correctness and unnecessary churn.

- A A successful merge, positive pull request emoji reactions, and no comments after the first hour.
- B Maximum number of replaced logging calls, shortest session duration, and the fewest prompt tokens used.
- C No behavior-test regressions, limited diff scope, fewer reviewer cleanup comments, and unchanged public API snapshots. ✓
- D A larger refactor that removes logging wrappers, updates dependencies, and rewrites service initialization.

✓ **Why the correct answer is right**

The stated intent is modernization without behavior changes and with less cleanup. The best criteria combine quantitative regression and API checks with qualitative reviewer feedback and diff-scope control.

✗ **Why the other options are wrong**

- **B:** Replacement count and speed can reward broad churn rather than safe, behavior-preserving modernization.
- **A:** Merge status and reactions are weak proxies and do not verify behavior or scope control.
- **D:** Expanding the task into a larger refactor conflicts with the intent to avoid behavior changes.

💡 Align evaluation criteria with intent: for safe repetitive changes, look for low churn, passing tests, stable interfaces, and reduced review rework.

Q62**Domain F: Guardrails, safety, accountability, and governance**

single choice

hard

A maintainer notices that an issue assigned to Copilot cloud agent contains a visible bug report and an HTML comment with instructions to exfiltrate repository files. A contractor without write access added the comment after the issue was created. What should the maintainer rely on first?

The team wants to keep using Copilot for normal issue work, but it must prevent untrusted prompt content from steering the agent.

- A** Move the repository to self-hosted runners so the hidden comment cannot reach the agent environment.
- B** Run the agent and inspect the final diff because PR review is enough to catch prompt injection effects.
- C** Use Copilot's trigger restrictions and hidden-character filtering, then review the session log and PR normally. ✓
- D** Add a repository secret containing a trusted system prompt that overrides user comments during execution.

✓ **Why the correct answer is right**

Copilot cloud agent limits who can trigger it, does not present comments from users without write access, and filters hidden content such as HTML comments before passing input to the agent. Normal review and session-log inspection remain part of accountability.

✗ **Why the other options are wrong**

- **B:** Reviewing only the final diff is reactive and ignores built-in input filtering and trigger restrictions.
- **A:** Changing runner infrastructure does not address malicious prompt text in GitHub issues or comments.
- **D:** A secret-based prompt override is not the documented control and risks exposing policy content in the wrong mechanism.

💡 Prompt-injection items often test whether you know the built-in input boundary. Do not replace it with unrelated runner or secret patterns.

prompt-injection

trigger-controls

built-in-protections

Q63

Domain B: MCP/tool use and environment permissions

single choice

hard

A DevOps team wants a nightly GitHub Actions workflow to ask Copilot to summarize the day's repository changes into the workflow summary. The job should be noninteractive and limited to reading git history and writing a local summary file. Which implementation is best?

The team is using GitHub Copilot CLI in CI, not Copilot cloud agent assigned from an issue.

- A Use GITHUB_TOKEN as the Copilot credential and omit tool flags for faster setup.
- B Run copilot -p with COPILOT_GITHUB_TOKEN, --allow-tool for git and write, and --no-ask-user. ✓**
- C Run Copilot CLI with --allow-all so it can decide which tools the summary needs.
- D Assign the default branch to Copilot each night and let it open a summary pull request.

✓ **Why the correct answer is right**

GitHub documents running Copilot CLI programmatically in Actions with `copilot -p`, `COPILOT_GITHUB_TOKEN`, explicit `--allow-tool` permissions, and `--no-ask-user` for noninteractive workflows. Allowing git and write matches the stated task.

✗ **Why the other options are wrong**

- **D:** This changes the workflow into a cloud-agent branch and PR task instead of a CI summary job.
- **C:** `--allow-all` grants broader tool access than the summary task requires and is discouraged outside sandbox scenarios.
- **A:** Copilot CLI authentication uses its documented credential variable, and omitting tool flags weakens permission control.

💡 When the scenario says Copilot CLI in Actions, answer with programmatic CLI patterns and scoped flags; do not substitute cloud-agent issue assignment.

ci-invocation

copilot-cli

actions-permissions

Q64**Domain C: Memory, state, and execution**

single choice

medium

A backend team repeatedly asks Copilot cloud agent and Copilot code review to follow the same repository-specific rule: every new repository class must use the existing `UnitOfWork` wrapper and the ``make test-integration`` command. The rule is documented in code patterns but not in a single policy file. What should the team use to reduce repeated prompting while keeping the information scoped to this repository?

The team wants later cloud-agent sessions and code reviews in the same repository to benefit from the learned rule without exposing it to unrelated repositories.

- A** Use Copilot Memory repository-level facts, supported by citations to current code in the repository. ✓
- B** Keep the rule only in the active chat context for each cloud-agent assignment.
- C** Store the rule as a user-level preference so code review can apply it for every reviewer.
- D** Add the rule to an external vector store shared across all repositories in the organization.

✓ **Why the correct answer is right**

Repository-level Copilot Memory is designed for coding conventions, architectural decisions, build commands, and project-specific rules. It is scoped to the same repository and can be used by cloud agent and code review when validated.

✗ **Why the other options are wrong**

- **C:** User-level preferences are personal and code review applies repository-level facts, not user-level preferences.
- **D:** A broad external store increases governance burden and risks cross-repository leakage for a repository-specific rule.
- **B:** Short-term chat context disappears after the session and would force the team to repeat the rule.

💡 For GH-600 memory questions, choose the smallest durable scope that matches the reuse need: session, repository/user memory, or external storage.

memory-strategy

copilot-memory

repository-facts

Q65

Domain A: Agent architecture and SDLC integration

single choice

hard

A repository requires one approving review before merge. A developer assigns a bug issue to Copilot cloud agent, reviews the resulting pull request, and approves it. The pull request remains blocked. What is the correct governance interpretation?

The developer who assigned the issue has write access and believes their approval should count because they understand the bug best.

- A The pull request is blocked because Copilot must mark it Ready for review before approvals count.
- B The initiator can review, but their approval does not count; another reviewer must approve before merge. ✓**
- C The approval is blocked because Copilot-created branches require two reviews regardless of branch rules.
- D The approval is ignored because Copilot pull requests can only be approved by repository administrators.

✓ **Why the correct answer is right**

When a user asks Copilot to create a pull request and branch protection requires approvals, the initiating user's approval does not count toward the required approvals. Someone else must approve.

✗ **Why the other options are wrong**

- **C:** Copilot does not create a universal two-review requirement; repository rules define the required count.
- **A:** Copilot cannot mark its own pull requests Ready for review, but that is not why the initiator approval is excluded.
- **D:** A separate eligible reviewer is required; the reviewer does not have to be an administrator.

💡 For Copilot PR governance, distinguish who can comment or review from whose approval counts toward protected-branch requirements.

pr-governance

human-review

branch-protection

Q66**Domain E: Multi-agent orchestration and incident response**

multi select

hard

A multi-agent workflow is repairing a flaky deployment. The validator agent stopped producing progress after running tests, while the fixer agent already pushed commits to its PR branch. Select TWO recovery actions that preserve useful work and support auditability.

The incident policy requires reviewers to understand what happened before any rerun or rollback.

A Restart or steer validation from the last known checkpoint rather than discarding fixer commits. ✓

B Disable session logging temporarily to reduce overhead during the recovery attempt.

C Inspect the stalled session log to determine the last completed tool call and validation state. ✓

D Merge the fixer PR immediately because the validator is the only stalled component.

E Delete the validator PR branch so the next run starts from a clean repository state.

✓ **Why the correct answer is right**

Session logs reveal the agent's approach, tool usage, and progress. Restarting or steering from a checkpoint preserves valid commits while addressing the stalled validation path.

✗ **Why the other options are wrong**

- **D:** Merging without completed validation bypasses the quality gate that the stalled agent was meant to provide.
- **E:** Deleting the branch removes potentially useful state and weakens the recovery trail.
- **B:** Disabling logging undermines the auditability required by the incident policy.

💡 For failure recovery, preserve evidence first, then resume from the smallest reliable checkpoint.

stalled-execution

partial-failure

session-logs

Q67**Domain A: Agent architecture and SDLC integration**

multi select

hard

A team is designing an autonomous agent workflow for dependency upgrades. The agent should keep delivery fast while reducing unbounded autonomy and silent failures. Select TWO controls the team should implement.

The repository has required status checks and CODEOWNERS for package manifests. The team wants agent activity to be inspectable without adding manual approval to every low-risk command.

- A Disable CI on Copilot branches to avoid blocking the agent during iterative dependency fixes.
- B Use required checks and CODEOWNERS review on package manifests before any upgrade merges. ✓
- C Move dependency-update decisions to an external ticket queue and sync results weekly.
- D Let the agent merge patch-level dependency updates when local tests pass in its environment.
- E Require the agent to publish a plan and test summary as PR artifacts before human review. ✓

✓ **Why the correct answer is right**

Publishing a plan and test summary creates inspectable artifacts, while required checks and CODEOWNERS review enforce human and automated gates at merge time.

✗ **Why the other options are wrong**

- **D:** Agent self-merge is excessive autonomy for dependency changes and bypasses PR governance.
- **A:** Disabling CI removes an evaluation signal and increases the chance of silent failure.
- **C:** An external queue can track work, but it reduces GitHub as the primary control plane for review.

💡 For autonomy questions, the best answers usually add targeted checkpoints and artifacts rather than approving every action or bypassing controls.

anti-patterns

autonomy-guardrails

human-in-the-loop

Q68**Domain C: Memory, state, and execution**

code or config artifact

hard

A team wants Copilot cloud agent to resume an interrupted database-migration task without repeating discovery. A developer proposes writing progress checkpoints from setup steps to ``tmp/agent-state.json``. Which change best aligns the artifact with durable state requirements?

The checkpoint must survive runner replacement and be visible to maintainers reviewing the agent's work.

 `.github/workflows/copilot-setup-steps.yml`

```
name: "Copilot Setup Steps"
on:
  workflow_dispatch:
  push:
    paths:
      - .github/workflows/copilot-setup-steps.yml
jobs:
  copilot-setup-steps:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - name: Record previous migration phase
        run: |
          echo '{"phase":"schema discovery","status":"complete"}' > /tmp/agent-state.json
```

- A Keep ``tmp/agent-state.json`` and add longer setup-step timeouts for interrupted agent sessions.
- B Persist the checkpoint in a PR comment, issue comment, or committed branch state file. ✓**
- C Move the checkpoint into a shell variable exported during setup-step execution.
- D Store the checkpoint in Copilot Memory so it becomes permanent task execution state.

✓ **Why the correct answer is right**

Copilot cloud agent works in an ephemeral GitHub Actions-powered development environment. Durable task state should be captured in reviewable GitHub artifacts such as PR comments, issue comments, or committed state files.

✗ **Why the other options are wrong**

- **A:** A longer timeout does not make ``tmp`` state durable across runner replacement.
- **C:** An environment variable is less durable than ``tmp`` and is not reviewable after interruption.
- **D:** Copilot Memory stores reusable facts and preferences, not per-run checkpoint state for task execution.

💡 Differentiate environment setup from durable workflow state; setup steps prepare execution, while checkpoints must live in GitHub-visible artifacts.

durable-artifacts

ephemeral-environment

execution-state

Q69

Domain A: Prepare agent architecture and SDLC processes

matching magnet

hard

A DevOps team is deciding which GitHub control to use for different risks introduced by agent-created pull requests. Match each requirement to the most appropriate GitHub control.

The team wants to use standard repository governance rather than building a separate approval portal.

Correct matches:

Prevent merging agent changes when tests or scans do not pass. → **Required status checks on the protected target branch**

Require platform-team approval when agent work edits critical workflow files. → **CODEOWNERS plus required code owner review**

Require manual approval before an agent-triggered deployment reaches production. → **Environment protection rules with required reviewers**

Apply interaction rules to branches whose names follow an agent-specific pattern. → **Branch or repository ruleset targeting the branch name pattern**

Choices: Environment protection rules with required reviewers, Required status checks on the protected target branch, CODEOWNERS plus required code owner review, Branch or repository ruleset targeting the branch name pattern, A longer issue template with more acceptance criteria, A scheduled audit report after pull requests are merged

✅ **Why the correct answer is right**

Required checks gate merge quality, CODEOWNERS routes sensitive file changes to responsible teams, environment protection gates deployments, and rulesets can apply rules to selected branch patterns.

💡 For governance matching, select the native GitHub control closest to the risk: merge quality, file ownership, deployment approval, or branch targeting.

pr-governance

branch-protection

environment-gates

Q70

Domain A: Prepare agent architecture and SDLC processes

sequence order

very_hard

A company wants Copilot cloud agent to run on self-hosted runners because the build requires private network access. In which order should the platform team introduce this execution environment safely?

The security team is concerned about runner reuse, open network paths, and untested setup steps.

Correct order:

1. Provision ephemeral, single-use Ubuntu x64 or Windows 64-bit runner capacity.
2. Configure network controls so the agent reaches only required internal and GitHub endpoints.
3. Add and validate copilot-setup-steps.yml before enabling agent work broadly.
4. Pilot the configuration on low-risk repositories and inspect session evidence.

✓ **Why the correct answer is right**

For self-hosted Copilot cloud agent runs, GitHub recommends ephemeral single-use runners, network controls, setup-step validation, and then a limited pilot before broad rollout.

✗ **Why the other options are wrong**

- **A-before-D:** Network policy should be attached to known runner capacity and supported runner types.
- **B-before-A:** Testing setup steps before network boundaries exist can hide whether the environment is safely constrained.
- **C-before-B:** A pilot before setup validation increases noise and makes failures harder to classify.

💡 Execution-environment architecture questions usually reward isolation first, then network boundaries, then validation, then broader adoption.

execution-environment

runner-isolation

network-boundaries

Q71

Domain A: Prepare agent architecture and SDLC processes

matching magnet

hard

A director asks each team to map agent responsibilities to SDLC stages before enabling autonomous work. Match each agent responsibility to the SDLC stage or control surface where it best belongs.

The organization wants a consistent operating model that uses GitHub issues, pull requests, checks, and deployment controls as the system of record.

Correct matches:

Convert a user-reported defect into scoped inputs and acceptance criteria.

→ **Planning and task definition**

Apply code changes on an isolated branch and open a pull request.

→ **Implementation through PR workflow**

Run tests, scans, and reviewer feedback before accepting the change.

→ **Validation and evaluation**

Gate promotion to production after the pull request has been accepted.

→ **Deployment control**

Choices: Deployment control, Planning and task definition, Implementation through PR workflow, Validation and evaluation, Persistent memory pruning, Custom agent repository setup

✓ **Why the correct answer is right**

Issue scoping belongs to planning, branch and PR work belongs to implementation, tests and review belong to validation, and production promotion belongs to deployment control.

💡 When asked to map agents to the SDLC, follow the GitHub control plane: issue for intent, PR for implementation, checks and review for validation, environments for deployment.

sdlc-stage-mapping

agent-responsibilities

standard-tooling

Q72**Domain A: Prepare agent architecture and SDLC processes**

sequence order

very_hard

A monorepo team wants an agent workflow that can be resumed after failures without repeating earlier analysis. In which order should the team design durable handoffs between planning, implementation, and evaluation?

The team has seen long-running agent sessions lose focus after dependency installation failures and repeated test retries.

Correct order:

1. Capture task inputs, constraints, and success criteria in the issue or prompt.
2. Persist the plan, assumptions, and selected files as a reviewable GitHub artifact.
3. Start implementation from the approved artifact and write changes to a pull request branch.
4. Record test results and unresolved failures in PR comments or check outputs.

✓ **Why the correct answer is right**

Reliable handoffs begin with a clear task contract, then persist the plan and assumptions, then implement from the approved artifact, and finally record evaluation evidence for continuation or review.

✗ **Why the other options are wrong**

- **A-before-C:** Implementing before the plan artifact is persisted makes recovery and review harder after failures.
- **D-before-A:** Evaluation artifacts depend on implementation and test execution.
- **C-before-B:** A durable plan needs the task inputs and success criteria as its foundation.

💡 For reliability questions, prefer workflows that externalize state into GitHub artifacts rather than relying on the agent's transient context.

workflow-handoffs

structured-artifacts

evaluation-loop

Q73**Domain C: Memory, state, and execution**

single choice

hard

A platform group split a monolith into ``checkout-api`` and ``pricing-api``. Copilot cloud agent learned a repository-level fact in ``checkout-api``: service-to-service clients must include the ``X-Trace-Id`` header. The group expects Copilot code review in ``pricing-api`` to apply the same learned fact automatically. What should the architect recommend?

The header rule is now an organization-wide integration standard, but it was originally learned from code in one repository.

- A Wait for the 28-day memory timer to synchronize the fact into related repositories.
- B Rely on user-level preferences so every reviewer carries the header rule across repositories.
- C Document the standard separately or use governed external memory; repository facts remain repo-scoped. ✓
- D Ask a write-access user in ``pricing-api`` to trigger memory creation from ``checkout-api`` citations.

✓ **Why the correct answer is right**

Repository-level facts can be used only in operations on the same repository. An organization-wide standard should be captured in an appropriate shared artifact or governed external memory rather than relying on cross-repository memory bleed.

✗ **Why the other options are wrong**

- **A:** The 28-day behavior is retention for unused entries, not cross-repository propagation.
- **D:** Repository facts require supporting citations in the same repository context where they are used.
- **B:** User-level preferences are personal and are not applied by Copilot code review.

💡 When a fact must cross repository boundaries, Copilot Memory repository scope is the limitation; choose shared documentation or external memory.

repository-scope

external-memory

cross-repository-continuity

Q74**Domain F: Guardrails, safety, accountability, and governance**

single choice

hard

A healthcare SaaS team wants Copilot cloud agent to prepare hotfix pull requests, but production deployment requires change-manager approval and access to production environment secrets. Which control should enforce the human decision point?

The team wants Copilot to keep producing fixes quickly, while ensuring production credentials are not available before approval.

- A** Use a protected production environment with required reviewers and prevent self-approval for deployment jobs. ✓
- B** Let Copilot merge the hotfix after CI succeeds, then require manual verification in the production service.
- C** Allow automatic production deployment from copilot branches, but require signed commits on those branches.
- D** Store production credentials as repository variables and ask Copilot to deploy only after a reviewer comments.

✓ **Why the correct answer is right**

A protected environment with required reviewers gates the deployment job before it proceeds or accesses environment secrets. Preventing self-approval strengthens the human-in-the-loop requirement for production changes.

✗ **Why the other options are wrong**

- **D:** A comment convention is not an enforceable deployment protection rule and repository variables are not the right boundary for production secrets.
- **B:** Copilot cloud agent cannot approve or merge its own pull request, and verification after deployment is too late for production approval.
- **C:** Signed commits improve traceability, but they do not provide production approval or protect environment secrets.

💡 For irreversible or production-sensitive actions, prefer GitHub environment protection rules over conventions, comments, or post-deployment monitoring.

production-deployment

human-in-the-loop

environment-secrets

Q75

Domain A: Prepare agent architecture and SDLC processes

sequence order

very_hard

A data platform group wants an agent to modify schema migration scripts, but the change can affect downstream services. In which order should the group enforce a planning-first workflow?

The agent may inspect the repository and produce recommendations, but it must not change files until the architecture owner approves the proposed approach.

Correct order:

1. Let the agent inspect context and post a structured migration plan.
2. Require the architecture owner to approve or reject the plan in GitHub.
3. Allow file changes only after approval, using a pull request branch.
4. Evaluate the pull request with migration tests, checks, and human review.

✓ **Why the correct answer is right**

The agent should first produce an inspectable plan, then wait for owner approval, then act through a pull request branch, and finally be evaluated by tests, checks, and review.

✗ **Why the other options are wrong**

- **A-before-C:** Approval is not meaningful until the structured plan exists.
- **B-before-A:** Changing files before plan approval violates the required planning and execution boundary.
- **D-before-B:** Pull request evaluation cannot occur until the agent creates reviewable changes.

💡 When a scenario says the agent may inspect but not change, the correct sequence usually permits read/planning first and gates write actions later.

planning-boundary

structured-plan

approval-gate

Q76**Domain E: Multi-agent orchestration and incident response****single choice****medium**

A repository team wants to add a database-migration specialist agent to an existing multi-agent workflow. Several Copilot sessions are already open for unrelated documentation and test tasks. What is the best rollout action?

The team wants the agent available for new database tasks while avoiding disruption to active sessions and preserving review history.

- A Replace the default cloud agent globally until all migration-related work is finished.
- B Cancel active sessions so they restart with the newly available database-migration agent.
- C Add a new agent profile under `.github/agents`, merge it, and use it for new sessions. ✓**
- D Edit every open Copilot PR to include the new agent instructions in its description.

✓ **Why the correct answer is right**

Repository custom agents are added as agent profile files under `.github/agents` and become available after the file is committed and merged; active work does not need to be disrupted.

✗ **Why the other options are wrong**

- **D:** PR descriptions are review artifacts, not the configuration source for adding a reusable custom agent.
- **B:** Cancelling unrelated sessions loses progress without being required for adding a new profile.
- **A:** Replacing the default agent is broader than needed and can affect unrelated task routing.

💡 Lifecycle questions favor additive, versioned changes for new sessions rather than disrupting in-flight work.

agent-lifecycle

custom-agent-rollout

workflow-continuity

Q77

Domain A: Prepare agent architecture and SDLC processes

sequence order

hard

A backend team wants Copilot cloud agent to implement a noncritical validation change, but the security lead requires the team to separate planning from execution and keep the workflow auditable in GitHub. In which order should the team structure the agent workflow?

The repository already requires pull requests and status checks. The team wants the agent to move quickly while avoiding unapproved code changes.

Correct order:

1. Create a GitHub issue that includes inputs, expected outputs, and merge success criteria.
2. Require the agent to publish a structured plan as an inspectable GitHub artifact.
3. Have a human validate the plan before allowing the agent to modify code.
4. Let the agent create a pull request, then evaluate CI results and review feedback.

✓ **Why the correct answer is right**

The safest architecture starts by defining the task contract, then makes the plan inspectable, then gates execution on human validation, and finally uses PRs, checks, and review as the evaluation surface.

✗ **Why the other options are wrong**

- **B-before-A:** A plan produced before inputs and success criteria are defined is likely to be vague or misaligned.
- **C-before-B:** A human cannot meaningfully validate an agent plan until the plan exists in an inspectable form.
- **D-before-C:** Letting the agent act before plan approval weakens the planning and execution boundary.

💡 For GH-600 sequencing items, look for the order that moves from task contract to inspectable plan to approval to PR-based execution.

sdhc-integration

plan-act-evaluate

human-approval

Q78**Domain F: Guardrails, safety, accountability, and governance**

multi select

medium

A risk committee asks which Copilot cloud agent protections are built in before the organization adds custom rulesets and environment protections. Select THREE statements the platform owner can accurately present.

The committee is deciding whether additional controls are needed for agent-created pull requests in repositories with sensitive application code.

- A Copilot can approve its own pull request after CodeQL and secret scanning complete successfully.
- B Copilot-generated code is checked with CodeQL, secret scanning, and advisory database dependency checks. ✓
- C Copilot commits are authored by Copilot, signed, and linked to session logs for traceability. ✓
- D Copilot can read GitHub Actions secrets when setup steps run in the copilot environment.
- E Copilot can merge its draft pull request if the human requester has repository administrator access.
- F Actions workflows from Copilot pull requests are blocked by default until a write user approves them. ✓

✓ **Why the correct answer is right**

GitHub documents built-in validation using CodeQL, secret scanning, and GitHub Advisory Database checks; Copilot commits are signed and traceable to session logs; and workflows in Copilot PRs are blocked by default until approved by a user with write access.

✗ **Why the other options are wrong**

- **A:** Copilot cloud agent cannot approve its own pull requests.
- **D:** Copilot cloud agent does not have access to GitHub Actions secrets or variables during runtime or setup steps.
- **E:** Draft pull requests created by Copilot must be reviewed and merged by a human; Copilot cannot merge them.

💡 Built-in protection questions mix true safety controls with exaggerated autonomy. Eliminate answers that let the agent approve, merge, or read Actions secrets.

built-in-protections

workflow-approval

traceability

Q79**Domain D: Evaluation, telemetry, error analysis, and tuning**

code or config artifact

hard

A mobile backend team finds that agent-created pull requests repeatedly pass unit tests but fail integration tests because the agent does not start the local emulator. The team wants to tune instructions so future sessions validate the same way developers do. Which replacement is best?

Reviewers confirmed in session logs that Copilot ran npm test but did not run the emulator-backed integration suite.

 `.github/copilot-instructions.md`

```
# Repository instructions
```

```
Use clear code and follow existing conventions.  
Run relevant tests before opening a pull request.  
Keep pull requests small when possible.
```

- A Remove testing guidance so the agent can infer the fastest validation path per task.
- B Specify the emulator startup command, integration test command, and changes that require them. ✓**
- C Add a note that reviewers may request integration tests if they believe the change needs them.
- D Ask Copilot to summarize which tests it skipped in the pull request description.

✓ **Why the correct answer is right**

The observed failure is caused by vague validation instructions. GitHub recommends using repository custom instructions to tell Copilot how to build, test, and validate changes, so the tuning should add explicit commands and triggering conditions.

✗ **Why the other options are wrong**

- **C:** Reactive reviewer requests still allow future sessions to miss the required validation initially.
- **A:** Removing guidance makes the validation path less predictable and does not address the failure.
- **D:** A skipped-test summary improves transparency but does not cause the agent to run the required tests.

💡 **Instruction-tuning answers should make the missing constraint executable and specific, not merely ask for better summaries.**

instruction-tuning

workflow-constraints

custom-instructions

Q80**Domain C: Memory, state, and execution**

code or config artifact

hard

An internal orchestration service stores a Copilot agent checkpoint in the feature branch so a replacement agent can resume after runner failure. The next agent reads the following file before making changes. Which improvement best prevents stale context and repeated work?

The branch has received human commits since the checkpoint was written, and the issue acceptance criteria are still the source of truth.

 `.github/agent-state/payment-retry.json`

```
{
  "issue": 842,
  "agentRunId": "run-17",
  "baseSha": "31b4c9a",
  "completed": [
    "located retry policy",
    "updated RetryPolicy defaults"
  ],
  "next": "add regression tests",
  "decision": "use exponential backoff with jitter"
}
```

- A** Append full tool-call logs to the JSON file so every command result remains in context.
- B** Move the checkpoint to the agent prompt so it travels with the next session.
- C** Compare ``baseSha`` with branch HEAD, then reconcile completed steps against the issue criteria. ✓
- D** Delete the checkpoint at startup and ask the replacement agent to rediscover prior decisions.

✓ **Why the correct answer is right**

A durable checkpoint should be validated against branch HEAD before acting, then re-anchored to the issue or acceptance criteria so the resumed agent avoids repeating completed work or drifting from current requirements.

✗ **Why the other options are wrong**

- **D:** Deleting the checkpoint discards durable state and increases the chance of redoing completed analysis.
- **A:** Keeping full tool logs bloats context and can preserve stale details after the step is complete.
- **B:** Prompt-only state is less durable than a branch artifact and can diverge from branch changes.

💡 State-resume questions usually reward checkpoint validation against the current branch and original task, not larger prompts or raw history.

state-persistence

resume-work

prevent-stale-context

Q81

Domain A: Prepare agent architecture and SDLC processes

matching magnet

very_hard

An architecture board is classifying agent actions by autonomy level. Match each action category to the most appropriate control pattern.

The board wants delivery speed for low-risk work while preserving human judgment for changes that affect security, production, or compliance.

Correct matches:

Read repository files and produce a proposed implementation plan. → **Allow autonomous planning with logged, inspectable output**

Modify code in a feature branch and open a draft pull request. → **Require PR review, checks, and branch protection before merge**

Change deployment workflow files or security-sensitive policy configuration. → **Require CODEOWNERS or designated owner approval before acceptance**

Promote changes into a protected production environment. → **Require environment approval before the deployment job proceeds**

Choices: Allow autonomous planning with logged, inspectable output, Require PR review, checks, and branch protection before merge, Require CODEOWNERS or designated owner approval before acceptance, Require environment approval before the deployment job proceeds, Allow direct default-branch commits when the agent is trusted, Disable session logging to reduce reviewer cognitive load

✓ **Why the correct answer is right**

Autonomy should increase only where risk is low. Planning can be autonomous when logged, code changes should go through PR governance, sensitive files need owner review, and production deployment needs an environment gate.

💡 Autonomy questions reward proportional controls: low-risk inspection can be fast, while sensitive writes and production actions need stronger human checkpoints.

autonomy-levels

guardrails

human-in-the-loop

Q82**Domain C: Memory, state, and execution**

single choice

medium

A developer with Copilot Pro tells Copilot CLI, "Prefer table-driven tests and avoid snapshot tests in my reviews." Later, Copilot code review on a team pull request does not apply that personal preference. What is the best explanation?

The repository has Copilot Memory enabled for the developer, and repository-level facts are working in cloud-agent sessions.

- A The preference expired immediately because user-level preferences are not retained across repositories.
- B Copilot code review applies repository-level facts, while user-level preferences are not applied there. ✓**
- C The developer needed repository admin access before personal preferences could affect later interactions.
- D Copilot CLI preferences are stored externally and cannot be used by any other Copilot feature.

✓ **Why the correct answer is right**

Copilot Memory can store user-level preferences, but Copilot code review uses repository-level facts only. Personal preferences may apply to that user's later interactions, but they are not applied during code review.

✗ **Why the other options are wrong**

- **A:** User-level preferences can be available to the same user across Copilot interactions, subject to retention behavior.
- **C:** Repository admin access is not the boundary described for using personal preferences in later interactions.
- **D:** Copilot Memory facts and preferences can be shared across supported Copilot features within documented limits.

💡 For feature-boundary questions, identify which memory type a feature consumes; user preference and repository fact behavior differ.

user-preferences

code-review-memory

feature-boundaries

Q83**Domain D: Evaluation, telemetry, error analysis, and tuning****single choice****medium**

A team delegates routine TypeScript bug fixes to Copilot cloud agent and wants to decide whether the workflow is production-ready after two sprints. The fixes usually touch application code, tests, and package manifests. Which evaluation approach best aligns success criteria with the development intent?

The engineering manager wants objective signals that reflect correctness, security, and review effort rather than only counting completed agent tasks.

- A** Track number of prompts submitted, number of branches created, session length, and agent response verbosity.
- B** Track merged pull requests only, then review defects reported after release during the next sprint.
- C** Track developer satisfaction with Copilot summaries and average time until the first draft pull request.
- D** Track test pass rate, coverage delta, open security findings, build status, and reviewer rework per pull request. ✓

✓ **Why the correct answer is right**

The best evaluation combines quantitative delivery and quality signals with security scan outcomes and human review effort. GitHub documents that Copilot works in pull requests, can run tests and linters in its environment, and exposes analysis details in session logs.

✗ **Why the other options are wrong**

- **A:** Prompt and session activity can help with cost or efficiency analysis, but they do not prove the code is correct or safe.
- **B:** Merged PR count and delayed defect review are lagging indicators and miss pre-merge quality and security signals.
- **C:** Developer satisfaction and first-draft speed are useful adoption signals but are incomplete for production readiness.

💡 For GH-600 evaluation questions, prefer signals that map directly to the task outcome: tests, scans, review effort, and workflow artifacts.

success-criteria

evaluation-signals

security-scanning

Q84

Domain A: Prepare agent architecture and SDLC processes

matching magnet

hard

A lead engineer is reviewing where custom agent profiles should live for different use cases. Match each intended scope to the correct profile location.

The organization uses Copilot cloud agent and Copilot CLI, and wants to avoid mixing personal, repository, and organization-level agent definitions.

Correct matches:

A repository-specific cloud agent available only to that project. → **.github/agents/ in the project repository**

An organization-level cloud agent available across organization repositories. → **agents/ in the organization's .github-private repository**

A user-level Copilot CLI custom agent for one developer's local use. → **~/.copilot/agents/ in the user's home directory**

A project-level Copilot CLI custom agent shared with repository contributors. → **.github/agents/ in the repository for CLI project agents**

Choices: .github/agents/ in the project repository, ~/.copilot/agents/ in the user's home directory, agents/ in the organization's .github-private repository, .github/agents/ in the repository for CLI project agents, .github/workflows/ in the target repository, CODEOWNERS in the default branch

✓ **Why the correct answer is right**

Repository custom agents use .github/agents, organization-level cloud agents are published from the agents directory in .github-private, and Copilot CLI supports project and user agent locations.

💡 Custom-agent placement questions test scope. Identify whether the agent is personal, project-scoped, or organization-scoped before choosing a path.

custom-agent-placement

repository-scope

organization-scope

Q85**Domain A: Agent architecture and SDLC integration**

multi select

hard

A platform team is updating its repository issue template for tasks that may be assigned to Copilot cloud agent. The goal is to make each agent run easier to evaluate during review. Select THREE fields that should be added to the template.

Recent agent pull requests were hard to judge because reviewers could not tell which files were in scope or which validation evidence was expected.

- A Reviewer vacation schedule and expected time zone for follow-up comments.
- B Explicit input context, including issue links, affected paths, branch, and constraints. ✓**
- C Preferred model name and maximum number of agent reasoning steps allowed.
- D A request that the agent avoid explaining implementation trade-offs in the pull request.
- E Expected outputs, including plan artifact, commit diff, tests run, and PR description. ✓**
- F Success criteria, including required checks, review ownership, and security scan expectations. ✓**

✓ **Why the correct answer is right**

Structured agent tasks should define inputs, outputs, and success criteria. Those fields let reviewers evaluate whether the agent stayed in scope and produced the expected evidence.

✗ **Why the other options are wrong**

- **C:** Model choice may matter operationally, but it is not a substitute for task inputs, outputs, and success criteria.
- **A:** Reviewer availability can help scheduling, but it does not make agent output easier to evaluate.
- **D:** Suppressing trade-off explanations reduces inspectability and undermines review quality.

💡 When the stem asks how to evaluate agent work, look for inputs, outputs, and measurable success criteria.

structured-output

plan-act-evaluate

inspectable-artifacts

Q86

Domain B: MCP/tool use and environment permissions

multi select

hard

A platform team is connecting Copilot cloud agent to an internal incident-tracking MCP server. The agent only needs to read ticket titles and statuses before proposing code fixes. Select TWO configuration choices the team should make.

The MCP tools run autonomously after configuration, and the organization wants to minimize the blast radius of a compromised prompt or server response.

- A Enable the server's "*" tool list during rollout, then narrow tools after observing usage.
- B Store the token as an Agents secret whose name starts with COPILOT_MCP_. ✓**
- C Use the server's OAuth browser authorization flow for the remote MCP connection.
- D Allowlist only the ticket-read and status-read MCP tools required by the agent task. ✓**
- E Store the tracker token as an Actions secret named TRACKER_TOKEN for workflow reuse.

✓ **Why the correct answer is right**

Copilot cloud agent can use configured MCP tools autonomously, so the safer configuration is to allowlist the specific read-only tools required. MCP variables and secrets for cloud agent must be Agents secrets or variables prefixed with COPILOT_MCP_.

✗ **Why the other options are wrong**

- **A:** Starting with "*" exposes unnecessary tools during the riskiest rollout period instead of enforcing least privilege upfront.
- **E:** Copilot cloud agent does not receive Actions secrets; MCP credentials must use Agents secrets or variables with the required prefix.
- **C:** Copilot cloud agent documentation states that remote MCP servers using OAuth authentication are not currently supported.

💡 When a question mentions autonomous MCP tool use, prefer preemptive allowlisting and the dedicated Agents secret path over broad rollout or shared CI secrets.

mcp-configuration

agent-secrets

least-privilege

 Case study — Case Study: FinEdge Lending Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

FinEdge is a regulated lending company that hosts its core loan-decision services in GitHub Enterprise Cloud. The platform organization wants to adopt GitHub Copilot cloud agent and custom agents to reduce routine refactoring work, update test coverage, and improve documentation for risk-model APIs.

Existing Environment

The main repository is named ``decision-engine``. The organization uses GitHub Copilot Enterprise. The default branch is protected and requires pull requests, two approving reviews, passing CI, dependency review, secret scanning, and CODEOWNERS review for ``.github/**``, ``.risk-models/**``, and deployment workflow files. The team has created an organization-owned ``.github-private`` repository with private visibility for custom agent testing.

Business Requirements

The first pilot agent, ``.risk-refactor``, must help senior engineers modernize old Java service code and add tests. The team wants the agent to work quickly, but all regulated code changes must remain reviewable in GitHub. Engineering managers want agent sessions, plans, pull requests, comments, workflow logs, and review outcomes to serve as the system of record.

Technical Requirements

For ambiguous or high-risk work, Copilot must first research the repository and produce a structured implementation plan before code changes begin. Engineers must be able to review and refine the plan, inspect the diff, and decide when to open a pull request. Routine low-risk fixes may create pull requests directly after an issue is assigned. The custom agent profile must use explicit tools rather than inheriting all available tools.

Security and Compliance Requirements

Copilot must not deploy software, approve its own pull requests, or bypass required reviews. Private package installation requires access to Artifactory using a token scoped only to ``.decision-engine``. The team must use Copilot cloud agent secrets and variables rather than GitHub Actions, Codespaces, or Dependabot secrets. Network access should remain restricted to the minimum required package and GitHub endpoints.

FinEdge wants to release the ``.risk-refactor`` custom agent only after the platform team validates it privately. In which order should the team complete the rollout?

The organization already chose private visibility for ``.github-private``, and the pilot should become organization-wide only after successful testing.

Correct order:

1. Create the organization `.github-private` repository from GitHub's custom agents template.
2. Create and merge the test profile under `.github/agents` in the `.github-private` repository.
3. Test from the agents tab, inspect session results, and iterate until the behavior is acceptable.
4. Move the validated agent profile into the `agents` directory and merge the release change.

✅ **Why the correct answer is right**

The documented organization rollout starts by preparing `.github-private`, then creating a test agent in `.github/agents`, testing it privately through Copilot agents sessions, and finally moving the profile to `agents` to make it available across the organization.

❌ **Why the other options are wrong**

- **A:** Testing cannot happen before the organization repository and test agent profile exist on the default branch.
- **B:** Moving to `agents` releases the custom agent broadly, so it should occur after private validation.
- **C:** The repository preparation is the prerequisite for storing and testing organization-level custom agents.
- **D:** The test profile must be created before session testing, but after the `.github-private` repository exists.

💡 For GH-600 rollout sequencing, identify the scope change. Test-only locations come before organization-wide release locations.

custom-agent-release

agent-sdlc

controlled-rollout

Q88

Domain A: Prepare agent architecture and SDLC processes

multi select

very_hard

 Case study — Case Study: FinEdge Lending Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

FinEdge is a regulated lending company that hosts its core loan-decision services in GitHub Enterprise Cloud. The platform organization wants to adopt GitHub Copilot cloud agent and custom agents to reduce routine refactoring work, update test coverage, and improve documentation for risk-model APIs.

Existing Environment

The main repository is named ``decision-engine``. The organization uses GitHub Copilot Enterprise. The default branch is protected and requires pull requests, two approving reviews, passing CI, dependency review, secret scanning, and CODEOWNERS review for ``.github/**``, ``.risk-models/**``, and deployment workflow files. The team has created an organization-owned ``.github-private`` repository with private visibility for custom agent testing.

Business Requirements

The first pilot agent, ``.risk-refactor``, must help senior engineers modernize old Java service code and add tests. The team wants the agent to work quickly, but all regulated code changes must remain reviewable in GitHub. Engineering managers want agent sessions, plans, pull requests, comments, workflow logs, and review outcomes to serve as the system of record.

Technical Requirements

For ambiguous or high-risk work, Copilot must first research the repository and produce a structured implementation plan before code changes begin. Engineers must be able to review and refine the plan, inspect the diff, and decide when to open a pull request. Routine low-risk fixes may create pull requests directly after an issue is assigned. The custom agent profile must use explicit tools rather than inheriting all available tools.

Security and Compliance Requirements

Copilot must not deploy software, approve its own pull requests, or bypass required reviews. Private package installation requires access to Artifactory using a token scoped only to ``.decision-engine``. The team must use Copilot cloud agent secrets and variables rather than GitHub Actions, Codespaces, or Dependabot secrets. Network access should remain restricted to the minimum required package and GitHub endpoints.

FinEdge must keep agent work auditable in GitHub while avoiding unnecessary approval delays for routine iterations. Select TWO controls that best meet the case study requirements.

The compliance team wants proactive guardrails for regulated code, but developers still need to iterate with Copilot without waiting for a security lead on every comment.

- A** Require protected pull requests, status checks, and CODEOWNERS review before agent changes can merge. ✓
- B** Require a compliance officer to approve every agent research prompt before Copilot can read repository files.
- C** Store agent design decisions in a shared spreadsheet and reference the sheet from each pull request.
- D** Allow Copilot to approve its generated pull requests when all required status checks pass.
- E** Have the agent produce plans, diffs, test results, and review requests as GitHub artifacts. ✓

✓ **Why the correct answer is right**

Branch protection, status checks, and CODEOWNERS reviews enforce review gates before merge. Plans, diffs, test results, PRs, comments, and review requests keep agent activity inspectable in standard GitHub tooling.

✗ **Why the other options are wrong**

- **B:** Approving every research prompt would slow low-risk discovery and does not target the higher-risk action point: merging regulated changes.
- **C:** An external spreadsheet weakens GitHub as the system of record and separates decisions from the code, PR, and review trail.
- **D:** Copilot cloud agent cannot approve or merge its own pull requests, and human review is required for protected work.

💡 GH-600 governance questions often reward controls at the PR and merge boundary. Avoid choices that move evidence outside GitHub or add blanket approvals that do not reduce material risk.

sdhc-governance

inspectable-artifacts

pr-based-control

Q89**Domain A: Prepare agent architecture and SDLC processes**

single choice

hard

 Case study — Case Study: FinEdge Lending Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

FinEdge is a regulated lending company that hosts its core loan-decision services in GitHub Enterprise Cloud. The platform organization wants to adopt GitHub Copilot cloud agent and custom agents to reduce routine refactoring work, update test coverage, and improve documentation for risk-model APIs.

Existing Environment

The main repository is named ``decision-engine``. The organization uses GitHub Copilot Enterprise. The default branch is protected and requires pull requests, two approving reviews, passing CI, dependency review, secret scanning, and CODEOWNERS review for ``.github/**``, ``.risk-models/**``, and deployment workflow files. The team has created an organization-owned ``.github-private`` repository with private visibility for custom agent testing.

Business Requirements

The first pilot agent, ``.risk-refactor``, must help senior engineers modernize old Java service code and add tests. The team wants the agent to work quickly, but all regulated code changes must remain reviewable in GitHub. Engineering managers want agent sessions, plans, pull requests, comments, workflow logs, and review outcomes to serve as the system of record.

Technical Requirements

For ambiguous or high-risk work, Copilot must first research the repository and produce a structured implementation plan before code changes begin. Engineers must be able to review and refine the plan, inspect the diff, and decide when to open a pull request. Routine low-risk fixes may create pull requests directly after an issue is assigned. The custom agent profile must use explicit tools rather than inheriting all available tools.

Security and Compliance Requirements

Copilot must not deploy software, approve its own pull requests, or bypass required reviews. Private package installation requires access to Artifactory using a token scoped only to ``.decision-engine``. The team must use Copilot cloud agent secrets and variables rather than GitHub Actions, Codespaces, or Dependabot secrets. Network access should remain restricted to the minimum required package and GitHub endpoints.

FinEdge needs the ``.risk-refactor`` pilot to analyze a poorly understood risk-model package before making changes. Which initiation approach best satisfies the requirement to separate planning from execution?

The package affects regulated loan decisions, and the task is too ambiguous to safely assign as a direct implementation issue.

- A** Start a cloud agent prompt to research and produce a structured plan, then approve implementation after review. ✓
- B** Create a long-running Actions workflow that invokes Copilot after every commit to the default branch.
- C** Assign the issue to Copilot immediately and rely on the created pull request to reveal the plan.
- D** Ask the agent to implement first, then request a separate design summary after CI completes.

✓ **Why the correct answer is right**

Starting with a cloud agent research and planning session keeps reasoning separate from action. GitHub documentation describes using Copilot cloud agent to research a repository, create a plan, refine it, review the diff, and create a pull request only when ready.

✗ **Why the other options are wrong**

- **C:** Assigning an issue to Copilot creates a pull request directly, which is suitable for lower-risk implementation work but not for an ambiguous regulated task requiring plan approval first.
- **D:** Implementing before planning reverses the required control point and makes the plan a post-hoc explanation rather than an approval artifact.
- **B:** A workflow triggered from default-branch commits does not provide the required human checkpoint before agent action and adds unnecessary automation risk.

💡 For GH-600 planning questions, prefer workflows that produce inspectable plans before code changes. Direct PR creation is useful, but it is not the safest choice for ambiguous or regulated work.

plan-act-evaluate

human-in-the-loop

structured-planning

 Case study — Case Study: FinEdge Lending Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

FinEdge is a regulated lending company that hosts its core loan-decision services in GitHub Enterprise Cloud. The platform organization wants to adopt GitHub Copilot cloud agent and custom agents to reduce routine refactoring work, update test coverage, and improve documentation for risk-model APIs.

Existing Environment

The main repository is named ``decision-engine``. The organization uses GitHub Copilot Enterprise. The default branch is protected and requires pull requests, two approving reviews, passing CI, dependency review, secret scanning, and CODEOWNERS review for ``.github/**``, ``.risk-models/**``, and deployment workflow files. The team has created an organization-owned ``.github-private`` repository with private visibility for custom agent testing.

Business Requirements

The first pilot agent, ``.risk-refactor``, must help senior engineers modernize old Java service code and add tests. The team wants the agent to work quickly, but all regulated code changes must remain reviewable in GitHub. Engineering managers want agent sessions, plans, pull requests, comments, workflow logs, and review outcomes to serve as the system of record.

Technical Requirements

For ambiguous or high-risk work, Copilot must first research the repository and produce a structured implementation plan before code changes begin. Engineers must be able to review and refine the plan, inspect the diff, and decide when to open a pull request. Routine low-risk fixes may create pull requests directly after an issue is assigned. The custom agent profile must use explicit tools rather than inheriting all available tools.

Security and Compliance Requirements

Copilot must not deploy software, approve its own pull requests, or bypass required reviews. Private package installation requires access to Artifactory using a token scoped only to ``.decision-engine``. The team must use Copilot cloud agent secrets and variables rather than GitHub Actions, Codespaces, or Dependabot secrets. Network access should remain restricted to the minimum required package and GitHub endpoints.

FinEdge's first agent run fails because private Java dependencies cannot be downloaded from Artifactory. Which remediation best meets the security and compliance requirements?

A developer proposes disabling the Copilot firewall and reusing an existing GitHub Actions secret so the setup step can authenticate to Artifactory.

- A** Create a repository variable named ``GITHUB_ARTIFACTORY_TOKEN`` and let setup steps read it during dependency installation.
- B** Create a selected-repository Agents secret, use it in ``copilot-setup-steps.yml``, and allow only required Artifactory access. ✓
- C** Disable the firewall for all Copilot sessions and rely on audit logs to identify unexpected network access.
- D** Grant Copilot access to existing Actions secrets and approve its workflow runs when the pull request opens.

✓ **Why the correct answer is right**

Copilot cloud agent has dedicated Agents secrets and variables, and those values can be used by scripts and tools in its environment, including ``copilot-setup-steps.yml``. Keeping access scoped to the required repository and network destination aligns with least privilege.

✗ **Why the other options are wrong**

- **A:** Secret and variable names cannot start with ``GITHUB_``, and a variable is not the right control for a sensitive token.
- **D:** Copilot cloud agent does not have access to GitHub Actions, Codespaces, or Dependabot secrets and variables.
- **C:** Disabling the firewall broadly is a reactive control failure and increases data exfiltration risk instead of scoping network access.

💡 When GH-600 questions mention private dependencies, separate identity from network control: use Agents secrets for credentials and narrow network access instead of weakening the whole environment.

agent-environment

secrets-governance

network-guardrails

 Case study — Case Study: Fabrikam Health Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Fabrikam Health builds patient scheduling, claims-routing, and analytics software. The platform engineering group is piloting GitHub Copilot coding agent and organization-level custom agents to reduce routine backlog work while maintaining health-data compliance controls. The pilot covers the ``care-portal``, ``claims-api``, and ``ops-automation`` repositories.

Existing Environment

The organization uses GitHub Enterprise Cloud with GitHub Copilot Business. Copilot coding agent is enabled only for the three pilot repositories. The ``care-portal`` repository uses required pull requests, required CI status checks, and CODEOWNERS review for ``.github/**``, ``.github/workflows/**``, ``src/auth/**``, ``db/migrations/**``, and ``deploy/**``. Copilot-created branches are expected to use the ``copilot/`` branch prefix and be reviewed through the normal pull request workflow.

Business Requirements

The organization wants agents to handle low-risk work such as documentation updates, test coverage improvements, lint fixes, and small accessibility defects. Higher-risk work, including authentication changes, database migrations, deployment workflow changes, and changes to agent instructions, must have a human-approved plan before implementation. Agent output must remain visible in standard GitHub artifacts such as issues, pull requests, comments, status checks, and logs.

Technical Requirements

Agent tasks must define clear inputs, expected outputs, and success criteria. For implementation work, the expected outputs are a pull request, commit diff, test results, and a PR description explaining the approach. For planning-only work, the expected output is a structured plan comment that includes affected files, risks, proposed tests, and a human approval checkpoint. Custom agents for the organization must be released from the organization's custom agent repository rather than copied independently into every product repository.

Security and Compliance Requirements

Agents must use least-privilege tools and avoid broad repository or external-service access. The platform team must preserve independent human review for agent-authored code. Copilot must not approve, merge, or mark its own pull requests ready for review. Compliance reviewers want a repeatable process that prevents silent failures, prevents unapproved high-risk actions, and limits stale or unrelated context from influencing later tasks.

Fabrikam's database team wants Copilot to help with a schema migration that affects PHI reporting. Which task design should the platform team implement?

Compliance says no code changes can occur until a human approves the plan, but approved implementation should still use the normal pull request workflow.

- A Assign the migration issue directly to Copilot, then require database owners to review only after Copilot opens its implementation PR.
- B Disable Copilot for migration work and require developers to manually copy any useful Copilot Chat suggestions into branches.
- C Use a planning-only task that posts a structured plan, then trigger implementation only after an approved comment or label. ✓**
- D Ask Copilot to open a draft PR with code and plan together, then block workflow runs until approval is granted.

✓ **Why the correct answer is right**

A planning-only task separates planning from execution and creates an inspectable GitHub artifact before code changes occur. Triggering implementation only after an approved comment or label satisfies the required human checkpoint while preserving PR-based implementation and evaluation.

✗ **Why the other options are wrong**

- **A:** This allows implementation before the required human-approved plan, reversing the required plan-before-act boundary.
- **D:** Combining code and plan in the same draft PR still permits code changes before plan approval, even if workflow runs are blocked.
- **B:** Disabling agent involvement avoids the risk but fails to use a controlled agent workflow that could satisfy the requirement.

💡 When a stem says action must wait for approval, look for a two-stage design: structured plan first, explicit approval second, implementation third.

plan-act-evaluate

structured-agent-task

approval-before-action

 Case study — Case Study: Fabrikam Health Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Fabrikam Health builds patient scheduling, claims-routing, and analytics software. The platform engineering group is piloting GitHub Copilot coding agent and organization-level custom agents to reduce routine backlog work while maintaining health-data compliance controls. The pilot covers the ``care-portal``, ``claims-api``, and ``ops-automation`` repositories.

Existing Environment

The organization uses GitHub Enterprise Cloud with GitHub Copilot Business. Copilot coding agent is enabled only for the three pilot repositories. The ``care-portal`` repository uses required pull requests, required CI status checks, and CODEOWNERS review for ``.github/**``, ``.github/workflows/**``, ``src/auth/**``, ``db/migrations/**``, and ``deploy/**``. Copilot-created branches are expected to use the ``copilot/`` branch prefix and be reviewed through the normal pull request workflow.

Business Requirements

The organization wants agents to handle low-risk work such as documentation updates, test coverage improvements, lint fixes, and small accessibility defects. Higher-risk work, including authentication changes, database migrations, deployment workflow changes, and changes to agent instructions, must have a human-approved plan before implementation. Agent output must remain visible in standard GitHub artifacts such as issues, pull requests, comments, status checks, and logs.

Technical Requirements

Agent tasks must define clear inputs, expected outputs, and success criteria. For implementation work, the expected outputs are a pull request, commit diff, test results, and a PR description explaining the approach. For planning-only work, the expected output is a structured plan comment that includes affected files, risks, proposed tests, and a human approval checkpoint. Custom agents for the organization must be released from the organization's custom agent repository rather than copied independently into every product repository.

Security and Compliance Requirements

Agents must use least-privilege tools and avoid broad repository or external-service access. The platform team must preserve independent human review for agent-authored code. Copilot must not approve, merge, or mark its own pull requests ready for review. Compliance reviewers want a repeatable process that prevents silent failures, prevents unapproved high-risk actions, and limits stale or unrelated context from influencing later tasks.

Fabrikam's compliance reviewers identify several agent design risks during the pilot readiness review. Match each risk to the best mitigation.

The platform team wants preventive controls that keep agent behavior inspectable without requiring reviewers to reconstruct decisions from unstructured logs.

Correct matches:

Silent failures in a security-remediation agent → **Fail visibly in GitHub by adding blocking status or check artifacts and escalation comments.**

Unbounded autonomy for deployment-related changes → **Require deterministic human approval before high-risk or irreversible actions proceed.**

Overlong or leaking context across unrelated service tasks → **Prune or reset task context and scope memory to the current repository task.**

No structured output for planning tasks → **Produce a structured plan, PR description, and test-result artifact for review.**

Choices: Require deterministic human approval before high-risk or irreversible actions proceed., Increase model temperature so the agent explores more implementation paths., Produce a structured plan, PR description, and test-result artifact for review., Fail visibly in GitHub by adding blocking status or check artifacts and escalation comments., Mirror production secrets into the runner so retries are less likely to fail., Prune or reset task context and scope memory to the current repository task.

✓ Why the correct answer is right

Silent failures require visible GitHub artifacts that block or escalate. High-risk autonomous actions require deterministic human approval. Context leakage and overlong context are mitigated by pruning, resetting, and scoping context. Planning tasks should produce structured artifacts that humans and CI can inspect.

💡 On GH-600 anti-pattern questions, map each risk to a control that changes system behavior, not to a vague instruction asking the model to be more careful.

agent-anti-patterns

observability

control-boundaries

 Case study — Case Study: Fabrikam Health Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Fabrikam Health builds patient scheduling, claims-routing, and analytics software. The platform engineering group is piloting GitHub Copilot coding agent and organization-level custom agents to reduce routine backlog work while maintaining health-data compliance controls. The pilot covers the ``care-portal``, ``claims-api``, and ``ops-automation`` repositories.

Existing Environment

The organization uses GitHub Enterprise Cloud with GitHub Copilot Business. Copilot coding agent is enabled only for the three pilot repositories. The ``care-portal`` repository uses required pull requests, required CI status checks, and CODEOWNERS review for ``.github/**``, ``.github/workflows/**``, ``src/auth/**``, ``db/migrations/**``, and ``deploy/**``. Copilot-created branches are expected to use the ``copilot/`` branch prefix and be reviewed through the normal pull request workflow.

Business Requirements

The organization wants agents to handle low-risk work such as documentation updates, test coverage improvements, lint fixes, and small accessibility defects. Higher-risk work, including authentication changes, database migrations, deployment workflow changes, and changes to agent instructions, must have a human-approved plan before implementation. Agent output must remain visible in standard GitHub artifacts such as issues, pull requests, comments, status checks, and logs.

Technical Requirements

Agent tasks must define clear inputs, expected outputs, and success criteria. For implementation work, the expected outputs are a pull request, commit diff, test results, and a PR description explaining the approach. For planning-only work, the expected output is a structured plan comment that includes affected files, risks, proposed tests, and a human approval checkpoint. Custom agents for the organization must be released from the organization's custom agent repository rather than copied independently into every product repository.

Security and Compliance Requirements

Agents must use least-privilege tools and avoid broad repository or external-service access. The platform team must preserve independent human review for agent-authored code. Copilot must not approve, merge, or mark its own pull requests ready for review. Compliance reviewers want a repeatable process that prevents silent failures, prevents unapproved high-risk actions, and limits stale or unrelated context from influencing later tasks.

Fabrikam is preparing to release reusable custom agents for the three pilot repositories. Select TWO actions the platform team should take.

The agents must be centrally governed at the organization level and released through the organization's custom agent repository rather than copied independently into each product repository.

- A Grant broad tool access in YAML front matter so each agent can adapt without repeated permission changes.
- B Store organization-level agent profiles in the root agents directory of .github-private before release testing. ✓**
- C Move every agent profile into each product repository so teams can edit prompts without an organization-level release process.
- D Let Copilot mark draft PRs ready when tests pass to reduce maintainer handoff time.
- E Create the organization .github-private repository from GitHub's custom agents template with internal or private visibility. ✓**

✓ **Why the correct answer is right**

Organization-level custom agents are prepared by creating the `.github-private` repository from GitHub's template with appropriate visibility. Organization-level agent profiles are released from the root agents` directory in that repository.`

✗ **Why the other options are wrong**

- **C:** Copying profiles into each product repository fragments governance and conflicts with the requirement for centrally governed organization-level agents.
- **A:** Broad tool access is an over-permissioning pattern; the case requires least-privilege tools and controlled agent behavior.
- **D:** Copilot cannot mark its own pull requests ready for review, and the case requires preserving human review checkpoints.

💡 For custom-agent setup questions, distinguish repository-level `.github/agents` profiles from organization-level .github-private/agents` profiles.`

custom-agents

organization-agent-repository

agent-release-process

 Case study — Case Study: Fabrikam Health Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Fabrikam Health builds patient scheduling, claims-routing, and analytics software. The platform engineering group is piloting GitHub Copilot coding agent and organization-level custom agents to reduce routine backlog work while maintaining health-data compliance controls. The pilot covers the ``care-portal``, ``claims-api``, and ``ops-automation`` repositories.

Existing Environment

The organization uses GitHub Enterprise Cloud with GitHub Copilot Business. Copilot coding agent is enabled only for the three pilot repositories. The ``care-portal`` repository uses required pull requests, required CI status checks, and CODEOWNERS review for ``.github/**``, ``.github/workflows/**``, ``src/auth/**``, ``db/migrations/**``, and ``deploy/**``. Copilot-created branches are expected to use the ``copilot/`` branch prefix and be reviewed through the normal pull request workflow.

Business Requirements

The organization wants agents to handle low-risk work such as documentation updates, test coverage improvements, lint fixes, and small accessibility defects. Higher-risk work, including authentication changes, database migrations, deployment workflow changes, and changes to agent instructions, must have a human-approved plan before implementation. Agent output must remain visible in standard GitHub artifacts such as issues, pull requests, comments, status checks, and logs.

Technical Requirements

Agent tasks must define clear inputs, expected outputs, and success criteria. For implementation work, the expected outputs are a pull request, commit diff, test results, and a PR description explaining the approach. For planning-only work, the expected output is a structured plan comment that includes affected files, risks, proposed tests, and a human approval checkpoint. Custom agents for the organization must be released from the organization's custom agent repository rather than copied independently into every product repository.

Security and Compliance Requirements

Agents must use least-privilege tools and avoid broad repository or external-service access. The platform team must preserve independent human review for agent-authored code. Copilot must not approve, merge, or mark its own pull requests ready for review. Compliance reviewers want a repeatable process that prevents silent failures, prevents unapproved high-risk actions, and limits stale or unrelated context from influencing later tasks.

Fabrikam wants Copilot coding agent to keep delivery moving for routine work while preventing unapproved changes to protected paths. Which governance design best satisfies the case study requirements?

The platform team must preserve normal GitHub pull request governance, avoid blanket approval gates, and create an auditable path for agent-authored changes.

- A Let Copilot push feature branches broadly, require post-merge audit review, and ask reviewers to revert protected-file changes if CI later fails.
- B Keep Copilot on copilot/* branches, require PRs, CI, CODEOWNERS for protected paths, and use an approval label before implementation for high-risk plans. ✓**
- C Disable Copilot for protected directories, allow direct merges for other changes, and review agent session logs during weekly compliance sampling.
- D Require platform approval for every Copilot-created PR, skip path-specific CODEOWNERS, and rely on reviewers to notice high-risk file changes.

✓ **Why the correct answer is right**

This design combines least-privilege agent execution with GitHub-native governance. Copilot coding agent work remains in pull requests, required checks and CODEOWNERS enforce acceptance controls, and high-risk work receives a deterministic approval checkpoint before implementation.

✗ **Why the other options are wrong**

- **A:** Post-merge review is reactive and does not prevent high-risk changes from entering the protected branch before compliance review.
- **C:** Blanket directory disablement reduces useful automation and weekly log sampling does not provide a timely control before risky changes merge.
- **D:** A universal platform approval gate slows low-risk delivery and skipping path-specific CODEOWNERS removes the most targeted review control.

💡 For GH-600 governance questions, favor controls that keep agent work inside GitHub PR workflows and add targeted human checkpoints only where risk justifies them.

pr-governance

human-in-the-loop

agent-autonomy-guardrails

 Case study — Case Study: Contoso Health Claims Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Contoso Health operates a GitHub Enterprise Cloud organization that hosts a regulated claims-processing platform. The organization wants to use Copilot cloud agent and custom agents to accelerate routine engineering work while preserving auditability, review discipline, and deployment safety.

Existing Environment

The platform is split across a monorepo named claims-platform and several service repositories. Copilot cloud agent is enabled only for selected repositories. The claims-platform repository uses branch protection on main, required pull requests, required status checks, and CODEOWNERS review for .github/**, database migrations, billing calculation code, and deployment workflows. The organization has created a .github-private repository from GitHub's custom agents template, but it has not yet released any organization-level custom agents.

Business Requirements

The engineering organization must reduce cycle time for well-scoped maintenance work such as test updates, documentation updates, small defect fixes, and dependency cleanup. Teams must be able to pilot specialized agents with a small reviewer group before making them broadly available. Product owners want issues to be assignable to Copilot only when the task has clear acceptance criteria and a bounded implementation scope.

Technical Requirements

For high-risk tasks, Copilot must research the repository and produce a structured implementation plan before changing files. The plan must identify the inputs, affected paths, expected outputs, validation steps, and success criteria. A human reviewer must approve the plan in GitHub before implementation begins. Agent work must result in inspectable artifacts in normal GitHub workflows, such as issue comments, pull request descriptions, commits, session logs, status checks, and review comments.

Security and Compliance Requirements

Agent changes must be traceable to the triggering user and the Copilot session. Copilot-created changes must be reviewed by humans before merge. Deployment to regulated environments requires an environment approval gate. Teams must avoid agent anti-patterns including silent failures, unbounded autonomy, stale context reuse, and prose-only outputs that are difficult to validate.

Contoso Health reviews three failed pilot runs before expanding Copilot cloud agent usage. Match each observed failure pattern to the most appropriate SDLC control.

The controls must preserve delivery speed while preventing unreviewable work, hidden failures, and context drift.

Correct matches:

A setup command fails, but the agent continues and omits generated clients from the pull request.

→ **Surface failures through inspectable logs, status checks, and escalation instead of swallowing them**

A broad optimization issue causes the agent to edit unrelated payment and deployment policy files.

→ **Require a bounded structured plan and human approval before implementation begins**

The agent uses old incident context and applies a workaround that the team retired last quarter.

→ **Prune context to current task artifacts and reset stale cross-task assumptions**

Choices: Expand the agent's repository token permissions so it can repair dependent deployment settings itself, Require a bounded structured plan and human approval before implementation begins, Surface failures through inspectable logs, status checks, and escalation instead of swallowing them, Allow the agent to merge after Copilot code review finishes to shorten the feedback cycle, Prune context to current task artifacts and reset stale cross-task assumptions

✅ Why the correct answer is right

The first observation is a silent-failure risk and needs visible logs, checks, and escalation. The second is unbounded autonomy and needs a structured plan with bounded scope before action. The third is stale context drift and needs context pruning or reset to current task artifacts.

💡 In GH-600 matching items about failed agent runs, classify the failure first: silent failure, excessive autonomy, context leakage, stale context, or missing structured output. Then choose the control that prevents that class of failure.

agent-anti-patterns

observability-and-control

context-boundaries

 Case study — Case Study: Contoso Health Claims Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Contoso Health operates a GitHub Enterprise Cloud organization that hosts a regulated claims-processing platform. The organization wants to use Copilot cloud agent and custom agents to accelerate routine engineering work while preserving auditability, review discipline, and deployment safety.

Existing Environment

The platform is split across a monorepo named claims-platform and several service repositories. Copilot cloud agent is enabled only for selected repositories. The claims-platform repository uses branch protection on main, required pull requests, required status checks, and CODEOWNERS review for .github/**, database migrations, billing calculation code, and deployment workflows. The organization has created a .github-private repository from GitHub's custom agents template, but it has not yet released any organization-level custom agents.

Business Requirements

The engineering organization must reduce cycle time for well-scoped maintenance work such as test updates, documentation updates, small defect fixes, and dependency cleanup. Teams must be able to pilot specialized agents with a small reviewer group before making them broadly available. Product owners want issues to be assignable to Copilot only when the task has clear acceptance criteria and a bounded implementation scope.

Technical Requirements

For high-risk tasks, Copilot must research the repository and produce a structured implementation plan before changing files. The plan must identify the inputs, affected paths, expected outputs, validation steps, and success criteria. A human reviewer must approve the plan in GitHub before implementation begins. Agent work must result in inspectable artifacts in normal GitHub workflows, such as issue comments, pull request descriptions, commits, session logs, status checks, and review comments.

Security and Compliance Requirements

Agent changes must be traceable to the triggering user and the Copilot session. Copilot-created changes must be reviewed by humans before merge. Deployment to regulated environments requires an environment approval gate. Teams must avoid agent anti-patterns including silent failures, unbounded autonomy, stale context reuse, and prose-only outputs that are difficult to validate.

Contoso Health wants to pilot a specialized claims-review custom agent with only five senior reviewers, then release the same agent across the organization without copying it

into each repository. Which approach should the platform team use?

The .github-private repository already exists. The pilot must be private, and the release process must use the supported organization-level custom agent layout.

- A** Test the profile in .github-private/.github/agents, then move it to the root agents directory for release. ✓
- B** Store the prompt in the .github-private README and ask reviewers to paste it into each Copilot task.
- C** Commit the agent directly to the root agents directory and depend on CODEOWNERS to limit pilot usage.
- D** Create identical repository-level agent files in each service repository, then delete the duplicates after the pilot.

✓ **Why the correct answer is right**

For organization or enterprise custom agents, GitHub documents a pilot path in the .github/agents directory of the .github-private repository, followed by release by moving the agent profile into the root agents directory. This avoids per-repository duplication while supporting controlled testing.

✗ **Why the other options are wrong**

- **D:** Per-repository copies increase drift and do not use the organization-level release mechanism.
- **C:** Putting the agent in the release directory first makes it broadly available before the pilot is complete.
- **B:** A README prompt is not a custom agent profile and does not provide managed agent selection or lifecycle control.

💡 For custom-agent lifecycle questions, distinguish repository-level agents from organization-level agents, and remember that .github-private supports a test location before release.

custom-agent-release-process

organization-agent-architecture

pilot-before-release

 Case study — Case Study: Contoso Health Claims Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Contoso Health operates a GitHub Enterprise Cloud organization that hosts a regulated claims-processing platform. The organization wants to use Copilot cloud agent and custom agents to accelerate routine engineering work while preserving auditability, review discipline, and deployment safety.

Existing Environment

The platform is split across a monorepo named claims-platform and several service repositories. Copilot cloud agent is enabled only for selected repositories. The claims-platform repository uses branch protection on main, required pull requests, required status checks, and CODEOWNERS review for .github/**, database migrations, billing calculation code, and deployment workflows. The organization has created a .github-private repository from GitHub's custom agents template, but it has not yet released any organization-level custom agents.

Business Requirements

The engineering organization must reduce cycle time for well-scoped maintenance work such as test updates, documentation updates, small defect fixes, and dependency cleanup. Teams must be able to pilot specialized agents with a small reviewer group before making them broadly available. Product owners want issues to be assignable to Copilot only when the task has clear acceptance criteria and a bounded implementation scope.

Technical Requirements

For high-risk tasks, Copilot must research the repository and produce a structured implementation plan before changing files. The plan must identify the inputs, affected paths, expected outputs, validation steps, and success criteria. A human reviewer must approve the plan in GitHub before implementation begins. Agent work must result in inspectable artifacts in normal GitHub workflows, such as issue comments, pull request descriptions, commits, session logs, status checks, and review comments.

Security and Compliance Requirements

Agent changes must be traceable to the triggering user and the Copilot session. Copilot-created changes must be reviewed by humans before merge. Deployment to regulated environments requires an environment approval gate. Teams must avoid agent anti-patterns including silent failures, unbounded autonomy, stale context reuse, and prose-only outputs that are difficult to validate.

Contoso Health is introducing Copilot cloud agent for database-adjacent claims changes that may affect regulated billing calculations. The architecture review board requires

planning to be separated from execution. In what order should the workflow be implemented?

The team wants Copilot to research and plan first, then act only after the plan has been checked and approved in GitHub.

Correct order:

1. Create an issue that states scope, affected paths, acceptance criteria, and asks Copilot for a structured plan only.
2. Have the domain owner validate the plan against requirements and explicitly authorize implementation in GitHub.
3. After approval, allow Copilot to implement on a Copilot branch and open a pull request with linked plan context.
4. Evaluate the pull request using CI, CODEOWNERS review, session logs, and the required human deployment gate.

✓ **Why the correct answer is right**

The correct sequence creates a bounded task, obtains a structured plan before action, requires human validation before implementation, and then evaluates the resulting PR through standard GitHub controls. This preserves the planning, action, and evaluation boundaries required by the case study.

✗ **Why the other options are wrong**

- **A:** Implementation belongs after plan validation, not before the domain owner has approved the approach.
- **B:** The task definition is the correct starting point because it gives the agent bounded inputs and success criteria.
- **C:** Evaluation occurs after the agent has produced a pull request and inspectable artifacts.
- **D:** Plan validation must occur before implementation, but it cannot occur until the structured plan exists.

💡 For GH-600 sequence questions, look for the governance boundary: define the task, require a plan, approve the plan, then permit action and evaluate through PR controls.

plan-act-evaluate

human-approval-gates

structured-agent-plans

 Case study — Case Study: Contoso Health Claims Platform

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Contoso Health operates a GitHub Enterprise Cloud organization that hosts a regulated claims-processing platform. The organization wants to use Copilot cloud agent and custom agents to accelerate routine engineering work while preserving auditability, review discipline, and deployment safety.

Existing Environment

The platform is split across a monorepo named claims-platform and several service repositories. Copilot cloud agent is enabled only for selected repositories. The claims-platform repository uses branch protection on main, required pull requests, required status checks, and CODEOWNERS review for .github/**, database migrations, billing calculation code, and deployment workflows. The organization has created a .github-private repository from GitHub's custom agents template, but it has not yet released any organization-level custom agents.

Business Requirements

The engineering organization must reduce cycle time for well-scoped maintenance work such as test updates, documentation updates, small defect fixes, and dependency cleanup. Teams must be able to pilot specialized agents with a small reviewer group before making them broadly available. Product owners want issues to be assignable to Copilot only when the task has clear acceptance criteria and a bounded implementation scope.

Technical Requirements

For high-risk tasks, Copilot must research the repository and produce a structured implementation plan before changing files. The plan must identify the inputs, affected paths, expected outputs, validation steps, and success criteria. A human reviewer must approve the plan in GitHub before implementation begins. Agent work must result in inspectable artifacts in normal GitHub workflows, such as issue comments, pull request descriptions, commits, session logs, status checks, and review comments.

Security and Compliance Requirements

Agent changes must be traceable to the triggering user and the Copilot session. Copilot-created changes must be reviewed by humans before merge. Deployment to regulated environments requires an environment approval gate. Teams must avoid agent anti-patterns including silent failures, unbounded autonomy, stale context reuse, and prose-only outputs that are difficult to validate.

Contoso Health notices that Copilot performs well on detailed issues but produces inconsistent pull requests when product owners assign vague cleanup tasks. Which issue-

template change best prepares tasks for agent execution?

The template must help product owners decide whether a task is safe to delegate and must give Copilot enough structure to produce reviewable work.

- A** Ask requesters to paste previous incident threads and let Copilot infer scope, tests, and success criteria.
- B** Let Copilot create the pull request first, then ask reviewers to define missing success criteria afterward.
- C** Keep issues brief and rely on repository custom instructions to supply implementation scope and review expectations.
- D** Require inputs, affected paths, expected outputs, validation commands, acceptance criteria, and prohibited files before assignment. ✓

✓ **Why the correct answer is right**

A structured task definition gives the agent clear inputs, bounded scope, outputs, validation steps, and success criteria before work begins. That aligns with the requirement to make delegated work reviewable and to avoid unbounded autonomy.

✗ **Why the other options are wrong**

- **A:** Large incident history can introduce stale or irrelevant context and still leaves the agent to infer success criteria.
- **C:** Custom instructions help with conventions, but they do not replace task-specific inputs, outputs, and acceptance criteria.
- **B:** Defining success criteria after implementation makes evaluation reactive and weakens the planning boundary.

💡 When a GH-600 scenario mentions vague agent outputs, prefer the option that defines task inputs, outputs, and measurable success criteria before execution.

structured-task-definition

inputs-outputs-success-criteria

sdlc-integration

Q99**Domain A: Prepare agent architecture and SDLC processes****single choice****very_hard** **Case study — Case Study: Orion Retail Platform Engineering**

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Orion Retail operates a GitHub Enterprise Cloud organization for its e-commerce platform. The platform engineering team is introducing GitHub Copilot cloud agent to reduce lead time for routine engineering work while keeping regulated payment, tax, and deployment changes under human governance. The team wants agents to behave like supervised contributors, not independent release authorities.

Existing Environment

The organization has Copilot Enterprise enabled. Copilot cloud agent is enabled in the checkout-service, tax-rules-service, inventory-api, and storefront-web repositories. The default branch in each repository is protected and requires pull requests, CI status checks, and at least one approving review. CODEOWNERS already covers application directories, but platform-owned files such as .github/workflows, .github/copilot-instructions.md, and agent profile files are not yet consistently owned. Production deployment workflows use GitHub Actions environments, but the current production environment does not prevent deployment self-review.

Business Requirements

Orion Retail wants agents to handle small implementation tasks, dependency hygiene, test fixes, and documentation updates. For compliance-sensitive tasks, agents must produce an inspectable plan before implementation. The plan must identify intended files, validation commands, expected outputs, and success criteria. Human intervention should be targeted to meaningful risk points so that low-risk delivery is not slowed by unnecessary approvals.

Technical Requirements

All agent activity must remain visible through GitHub issues, pull requests, comments, commits, checks, and session logs. Agent work must follow a plan, act, evaluate lifecycle. Copilot-generated implementation work must be delivered through pull requests with CI evidence. The platform team is also preparing an organization-level custom security-remediation agent. They will pilot the agent with AppSec engineers before releasing it broadly to Orion Retail developers.

Security and Compliance Requirements

Agent-generated changes to production deployment workflows, custom agent profiles, Copilot setup steps, and repository-wide Copilot instructions require platform review. Copilot must not be treated as a merge authority or production deployment approver. Production deployment must require a protected-environment approval from a human reviewer other than the user who initiated the deployment. Audit reviewers must be able to trace why Copilot made a change by using GitHub-native evidence.

Orion Retail discovers that a Copilot-generated PR can modify application code that later flows into the production deployment workflow. Which control best enforces the case study's deployment requirement without blocking normal agent implementation work?

The platform team wants Copilot to continue opening PRs and running validation checks, but production deployment must require a human approval that the initiating user cannot self-approve.

- A Remove the deployment workflow from Copilot branches and ask humans to cherry-pick validated changes into release branches.
- B Use a protected production environment with required reviewers and prevent self-review ✓** for deployments, while keeping PR checks required.
- C Disable the agent firewall exception for the cloud provider so Copilot cannot reach deployment endpoints during coding.
- D Monitor session logs for production-deploy attempts and require release managers to stop any agent session that appears unsafe.

✓ **Why the correct answer is right**

A protected environment with required reviewers creates the human deployment gate at the production boundary. Enabling prevention of self-review ensures the user who initiated the deployment cannot approve that deployment job, while PR checks still allow agent implementation work to proceed through normal validation.

✗ **Why the other options are wrong**

- **D:** Monitoring session logs is useful for auditability, but it reacts after risk appears instead of enforcing a deployment gate.
- **A:** Cherry-picking adds manual release complexity and can separate deployed code from the reviewed PR evidence.
- **C:** Network restrictions reduce runtime access risk, but they do not enforce a protected production approval workflow.
- **B:** This option directly satisfies the required human gate at the deployment boundary.

💡 For autonomous-agent deployment questions, enforce control at the irreversible action boundary rather than disabling useful coding or validation workflows.

environment-protection

human-in-the-loop

deployment-governance

 Case study — Case Study: Orion Retail Platform Engineering

This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided. To answer the questions included in a case study, you will need to reference information that is provided in the case study. Each question is independent of the other questions in this case study.

Overview

Orion Retail operates a GitHub Enterprise Cloud organization for its e-commerce platform. The platform engineering team is introducing GitHub Copilot cloud agent to reduce lead time for routine engineering work while keeping regulated payment, tax, and deployment changes under human governance. The team wants agents to behave like supervised contributors, not independent release authorities.

Existing Environment

The organization has Copilot Enterprise enabled. Copilot cloud agent is enabled in the checkout-service, tax-rules-service, inventory-api, and storefront-web repositories. The default branch in each repository is protected and requires pull requests, CI status checks, and at least one approving review. CODEOWNERS already covers application directories, but platform-owned files such as .github/workflows, .github/copilot-instructions.md, and agent profile files are not yet consistently owned. Production deployment workflows use GitHub Actions environments, but the current production environment does not prevent deployment self-review.

Business Requirements

Orion Retail wants agents to handle small implementation tasks, dependency hygiene, test fixes, and documentation updates. For compliance-sensitive tasks, agents must produce an inspectable plan before implementation. The plan must identify intended files, validation commands, expected outputs, and success criteria. Human intervention should be targeted to meaningful risk points so that low-risk delivery is not slowed by unnecessary approvals.

Technical Requirements

All agent activity must remain visible through GitHub issues, pull requests, comments, commits, checks, and session logs. Agent work must follow a plan, act, evaluate lifecycle. Copilot-generated implementation work must be delivered through pull requests with CI evidence. The platform team is also preparing an organization-level custom security-remediation agent. They will pilot the agent with AppSec engineers before releasing it broadly to Orion Retail developers.

Security and Compliance Requirements

Agent-generated changes to production deployment workflows, custom agent profiles, Copilot setup steps, and repository-wide Copilot instructions require platform review. Copilot must not be treated as a merge authority or production deployment approver. Production deployment must require a protected-environment approval from a human reviewer other than the user who initiated the deployment. Audit reviewers must be able to trace why Copilot made a change by using GitHub-native evidence.

Orion Retail is preparing the organization-level security-remediation custom agent described in the case study. Put the steps in the order the platform team should use to pilot and then release the agent.

The agent must be tested privately by AppSec engineers before it becomes available to developers across the organization.

Correct order:

1. Create the organization's .github-private repository from GitHub's custom agents template with appropriate visibility.
2. Create the test agent profile in .github-private/.github/agents and merge it into the default branch.
3. Run pilot prompts from the .github-private repository and review recent sessions before changing release scope.
4. Move the approved profile into the root agents directory and merge to make it available across Orion Retail.

✅ **Why the correct answer is right**

The organization first needs the .github-private custom agent repository. For a private pilot, the test profile belongs under .github-private/.github/agents and must be merged to the default branch. The team should then test sessions and inspect results. After approval, moving the profile to the root agents directory and merging releases it broadly.

❌ **Why the other options are wrong**

- **A:** Creating the profile before the organization custom-agent repository exists skips the required setup location.
- **B:** Moving the profile to the root agents directory releases it broadly, so this belongs after the pilot succeeds.
- **C:** This is the first enabling step because organization-level custom agents are prepared through .github-private.
- **D:** Testing belongs after the test profile is merged and before the release-scope change.

💡 For custom-agent lifecycle questions, distinguish private testing location from organization-wide release location.

custom-agents

release-governance

organization-agent-setup

Quick Answer Key

#	Type	Correct Answer
Q1	code or config artifact	D
Q2	matching magnet	row1 → Human listed as co-author on the Copilot-authored commit; row2 → Copilot session logs showing reasoning and tool usage; row3 → Signed commits that appear as Verified on GitHub; row4 → Commit message links to the session log
Q3	multi select	B, E
Q4	multi select	D, F, C
Q5	code or config artifact	B
Q6	sequence order	D → A → B → C
Q7	matching magnet	row1 → Planning; row2 → Reasoning; row3 → Action; row4 → Evaluation
Q8	single choice	B
Q9	multi select	C, A
Q10	multi select	C, B, D
Q11	single choice	B
Q12	code or config artifact	A
Q13	multi select	E, C
Q14	code or config artifact	A
Q15	multi select	B, C
Q16	code or config artifact	B
Q17	single choice	A

#	Type	Correct Answer
Q18	single choice	B
Q19	code or config artifact	D
Q20	single choice	B
Q21	code or config artifact	D
Q22	single choice	B
Q23	code or config artifact	D
Q24	multi select	E, D
Q25	single choice	B
Q26	sequence order	A → B → C → D
Q27	matching magnet	row1 → Add escalation paths that surface failures in logs, PR comments, or checks; row2 → Scope memory and context to the current task and prune stale information; row3 → Require structured outputs such as plans, PR descriptions, and test evidence; row4 → Insert human approval before irreversible or compliance-sensitive actions
Q28	code or config artifact	A
Q29	multi select	E, A, D
Q30	single choice	B
Q31	sequence order	A → B → C → D
Q32	sequence order	C → A → D → B
Q33	code or config artifact	D

#	Type	Correct Answer
Q34	multi select	A, E
Q35	single choice	B
Q36	multi select	A, E
Q37	single choice	C
Q38	single choice	D
Q39	single choice	A
Q40	code or config artifact	A
Q41	single choice	B
Q42	matching magnet	row1 → Inputs; row2 → Outputs; row3 → Success criteria; row4 → Out-of-scope boundaries
Q43	code or config artifact	A
Q44	single choice	D
Q45	multi select	F, A
Q46	multi select	A, D
Q47	single choice	B
Q48	code or config artifact	D
Q49	code or config artifact	C
Q50	single choice	D
Q51	code or config artifact	A

#	Type	Correct Answer
Q52	sequence order	B → C → A → D
Q53	multi select	A, B
Q54	multi select	B, D, E
Q55	code or config artifact	B
Q56	code or config artifact	D
Q57	single choice	C
Q58	matching magnet	row1 → Silent failure; row2 → Context leakage; row3 → Unbounded autonomy; row4 → No structured output
Q59	single choice	B
Q60	code or config artifact	C
Q61	single choice	C
Q62	single choice	C
Q63	single choice	B
Q64	single choice	A
Q65	single choice	B
Q66	multi select	C, A
Q67	multi select	E, B
Q68	code or config artifact	B
Q69	matching magnet	row1 → Required status checks on the protected target branch; row2 → CODEOWNERS plus required code owner review; row3 → Environment

#	Type	Correct Answer
		protection rules with required reviewers; row4 → Branch or repository ruleset targeting the branch name pattern
Q70	sequence order	D → A → B → C
Q71	matching magnet	row1 → Planning and task definition; row2 → Implementation through PR workflow; row3 → Validation and evaluation; row4 → Deployment control
Q72	sequence order	B → C → A → D
Q73	single choice	C
Q74	single choice	A
Q75	sequence order	C → A → B → D
Q76	single choice	C
Q77	sequence order	A → B → C → D
Q78	multi select	B, C, F
Q79	code or config artifact	B
Q80	code or config artifact	C
Q81	matching magnet	row1 → Allow autonomous planning with logged, inspectable output; row2 → Require PR review, checks, and branch protection before merge; row3 → Require CODEOWNERS or designated owner approval before acceptance; row4 → Require environment approval before the deployment job proceeds
Q82	single choice	B
Q83	single choice	D
Q84	matching magnet	row1 → .github/agents/ in the project repository; row2 → agents/ in the organization's .github-private repository; row3 → ~/.copilot/agents/ in the user's home directory; row4 → .github/agents/ in the repository for CLI project agents

#	Type	Correct Answer
Q85	multi select	B, E, F
Q86	multi select	D, B
Q87	sequence order	C → D → A → B
Q88	multi select	A, E
Q89	single choice	A
Q90	policy control selection	B
Q91	single choice	C
Q92	matching magnet	row1 → Fail visibly in GitHub by adding blocking status or check artifacts and escalation comments.; row2 → Require deterministic human approval before high-risk or irreversible actions proceed.; row3 → Prune or reset task context and scope memory to the current repository task.; row4 → Produce a structured plan, PR description, and test-result artifact for review.
Q93	multi select	E, B
Q94	policy control selection	B
Q95	matching magnet	row1 → Surface failures through inspectable logs, status checks, and escalation instead of swallowing them; row2 → Require a bounded structured plan and human approval before implementation begins; row3 → Prune context to current task artifacts and reset stale cross-task assumptions
Q96	single choice	A
Q97	sequence order	B → D → A → C
Q98	single choice	D
Q99	single choice	B
Q100	sequence order	C → A → D → B